



DEPARTEMENT INFORMATIQUE

RAPPORT DE PROJET DE FIN D'ETUDES

En vue de l'obtention du:
Diplôme National d'Ingénieur Informatique
Option : Génie Logiciel-Architecture Logicielle

Cash Analytics Platform

Elaboré par :

Farhat Haythem

Soutenu le 15/07/2021 devant le jury :

Président : M. CHTIOUI Houssemeddine

ISSAT de Sousse

Examinatrice : Mme. KHEMAJA Maha

ISSAT de Sousse

Encadrant : M. MTIBAA Riadh

ISSAT de Sousse

Encadreur
Industriel : M. BELGUESMI Nabil

Business & Ai

Année Universitaire : 2020 / 2021

Code Sujet: FI-GL21-063

Remerciement

À l'issue de ce travail, je remercie le bon Dieu, tout puissant, de m'avoir donné la force pour survivre, ainsi que l'audace pour dépasser toutes les difficultés.

À mon encadrant académique M. Riadh MTIBAA pour sa bonne volonté d'accepter de m'encadrer, pour tout le temps qu'il m'a octroyé et pour tous les conseils qu'il m'a prodigués.

Je tiens également à exprimer mon sincère gratitude et ma reconnaissance envers toutes les personnes qui m'ont aidé ou ont participé de près ou de loin au bon déroulement de ce projet.

Table des matières

Table des figures	vi
-------------------	----

Liste des tableaux	viii
--------------------	------

1 Cadre général du projet	4
1.1 Présentation de l'organisme d'accueil	4
1.2 Cadre général du projet	5
1.3 Étude de l'existant	5
1.4 Solution proposée	6
1.4.1 Avantages de l'architecture micro-services	7
1.4.2 Les défis de l'architecture des micro-services	8
1.5 Objectifs	8
1.6 Méthodologie adoptée	9
1.6.1 Les méthodologies agiles	9
1.6.2 Kanban	9
1.6.3 Visualiser le flux de travail	10
1.6.4 Limiter les travaux en cours	10
1.6.5 Processus explicites	11
1.6.6 Livraison continue	11
2 Spécifications et architecture globale	12
2.1 Spécification des besoins	12
2.2 Les acteurs	12
2.3 Besoins fonctionnels	13
2.4 Besoins non fonctionnels	13
2.5 Diagrammes de cas d'utilisation	13
2.5.1 Cas d'utilisations globaux	13
2.5.2 Tableau de bord analytique	15
2.5.3 Gestion des pipelines de traitement des données	16
2.6 Surveillance d'infrastructure	17
2.7 Architecture globale du système	17
2.8 La Virtualisation d'environnement d'exécution	19
2.8.1 Introduction à Docker	19

3	Récupération et traitement des données	22
3.1	Description du processus	22
3.1.1	Les données des clients	22
3.1.2	Étapes du traitement des données	23
3.2	Étude et spécifications	23
3.2.1	Solutions Existantes	24
3.2.2	Introduction à Airflow	26
3.2.3	Architecture	27
3.2.4	Résolution des problèmes liés à l'utilisation de fichiers CSV	28
3.3	Implémentation	29
4	Tableau de bord analytique	32
4.1	Architecture Logique du microservice de tableau de bord	32
4.2	Diagramme de classes	34
4.3	Caching et performance	36
4.3.1	Base de données volatile : Redis	36
4.4	Réalisation	37
5	Création des chaînes CI/CD	42
5.1	Les pratiques DevOps	42
5.2	Outils	43
5.2.1	Git et GitHub	43
5.2.2	Serveur de CI/CD	45
5.2.3	DockerHub	45
5.3	Pipeline CI/CD d'un microservice	45
5.4	Migrations automatiques des bases de données	47
5.4.1	Problématique	48
5.4.2	Solution proposée	48
5.4.3	Outil de migration FlywayDb	48
5.5	Pipeline CI/CD de migration automatique de base de données	49
6	Surveillance de l'infrastructure	52
6.1	L'importance de la surveillance	52
6.2	Besoins fonctionnelles	52
6.3	Besoins non-fonctionnelles	52
6.4	Solutions existantes	53

6.5	Architecture Prometheus	55
6.6	Réalisation	56
7	Sécurité des microservices	60
7.1	Problématique	60
7.1.1	Défis de sécurité dans une architecture micro-service	60
7.1.2	Anti-pattern d'implémentation	60
7.2	Solution proposée	61
7.3	Conception	62
7.4	Outils	63
7.4.1	Json Web Token (JWT)	64
7.4.2	Signatures numériques	65
7.4.3	Hachage cryptographique	65
7.4.4	HTTP Cookies	65
7.5	Mécanismes d'authentification	65
7.5.1	Access token	66
7.5.2	Refresh token	66
7.5.3	Processus de connexion	67
7.5.4	Processus d'actualisation de jeton d'accès	68
7.5.5	Processus de validation de requêtes	69
7.5.6	Processus de déconnexion	70
7.6	Atténuation des attaques	70
7.6.1	Cross Site Scripting (XSS)	71
7.6.2	Cross-site Request Forgery (CSRF)	72
7.7	Implémentation	73
7.7.1	Environnement de développement	73
7.7.2	Réalisation	74

Table des figures

1.1	Logo de la société	4
1.2	Domaines d'activités de la société	5
1.3	Différence entre les architectures [1]	7
1.4	Un exemple de tableau Kanban	10
2.1	Diagramme de cas d'utilisation global	14
2.2	Cas d'utilisation du tableau de bord analytique	15
2.3	Diagramme CU de la gestion des pipelines de traitement des données . . .	16
2.4	Diagramme CU de la surveillance d'infrastructure	17
2.5	Architecture logique globale	18
2.6	Déploiement de conteneurs logiciels	20
3.1	Exemple d'un DAG	26
3.2	Architecture logique de Airflow	27
3.3	Schéma de la base de données des traitements	28
3.4	Liste des DAGs	29
3.5	Le dossier contenant les DAGs	29
3.6	Exemple d'un DAG	30
3.7	Journal d'exécution d'une tâche	30
3.8	Journal d'exécution d'une tâche	31
4.1	Architecture Logique du MS du tableau de bord	33
4.2	Diagramme de classes du micro-service tableau de bord	34
4.3	Diagramme de séquence d'une opération d'exécution de requête	37
4.4	Tableau de bord spécifique aux opérations de remboursements	38
4.5	Le tableau de bord spécifique aux opérations de retrait hebdomadaires - 1	38
4.6	Le tableau de bord spécifique aux opérations de retrait hebdomadaires - 2	39
4.7	le tableau de bord relatif aux opérations de remboursement mensuelles . .	40
4.8	le tableau de bord spécifique aux opérations de retrait annuel	41
5.1	La pratique DevOps	42
5.2	Représentation graphique de l'évolution d'un référentiel git	44
5.3	Logo de GitHub	44
5.4	Diagramme d'activité d'une chaîne de CI/CD de micro-service	46

5.5	Capture d'écran du log d'exécution d'un pipeline CI/CD pour un micro-service	47
5.6	logo de FlywayDb	48
5.7	Nomenclature des fichiers de migration	49
5.8	Historique des migrations	49
5.9	Diagramme d'activité du pipeline CI/CD de la migration de base de données	50
5.10	Log du pipeline auto-migration	51
6.1	Architecture détaillée du système de surveillance	55
6.2	Exemple de requête PromQL	57
6.3	Visualisation de la requête de la figure 6.2	57
6.4	Tableau de bords de surveillance du serveur de production	58
6.5	Tableau de bords de surveillance de la base de donnée principale	58
6.6	Tableau de bords de surveillance des conteneurs Docker	59
7.1	Diagramme d'entité-relation du service d'authentification	62
7.2	Décodage d'un token JWT	64
7.3	Diagramme de séquence de la connexion	67
7.4	Diagramme de séquence d'actualisation de jeton d'accès	68
7.5	Diagramme de séquence de validation de requête	69
7.6	Diagramme de séquence de déconnexion	70
7.7	Les étapes d'une attaque XSS	71
7.8	Les étapes d'une attaque CSRF	72
7.9	Fonctionnalités offertes par le service d'authentification	74
7.10	Diagramme de déploiement partiel de la plateforme	75

Liste des tableaux

3.1	Comparaison des outils de gestion de flux de travail open source populaires	25
4.1	Tableau explicatif des classes principales du micro-service tableau de bord	36
6.1	Tableau comparatif des solutions de surveillance	54
7.1	Tableau descriptif des entités du système	63

Acronyme

AI Artificial Intelligence

API Application Programming Interface

ATM Automated Teller Machine

CSRF Cross-Site Request Forgery

CPU Central Processing Unit

CSV Comma Separated Values

CI/CD Continuous Integration / Continuous Deployment

DAB Distributeur Automatique de Billets

DAG Directed Acyclic Graph

FTP File Transfer Protocol

HTTP Hypertext Transfer Protocol

IT Information Technology

JSON JavaScript Object Notation

JWT Json Web Token

MS Micro-service

XSS Cross-Site Scripting

Introduction Générale

Le marché des logiciels est un marché très concurrentiel et dynamique. Les demandes d'utilisateurs évoluent constamment et les développeurs doivent toujours mettre à jours leurs produits pour garantir leurs succès. À cause de ces changements l'industrie de génie logiciel s'est développée, de nouvelles méthodes et techniques sont apparues qui tentent de minimiser les délais de mise sur le marché sans sacrifier la qualité.

C'est dans ce cadre que s'inscrit notre stage de fin d'études en vue de l'obtention du diplôme national d'ingénieur en informatique, spécialité génie logiciel, au sein de l'Institut Supérieur des Sciences Appliquées et de Technologie de Sousse. Ce stage de quatre mois est effectué au sein de la start-up « Business & Ai », société de service en informatiques spécialisée dans le développement de solutions d'intelligence artificielle.

Notre objectif est de concevoir et prototyper une architecture micro-service qui aidera la migration d'une application monolithique existante. Cette application est une plate-forme Web d'analyse de trésorerie, alimentée par les technologies d'IA propriétaire de Business&Ai pour la prévision et l'optimisation des flux de cash. Nous souhaitons créer une infrastructure solide qui permettra à une petite équipe de développeurs et analystes de données de développer, tester et déployer rapidement un produit facilement extensible et scalable.

Ce rapport est une tentative de rendre compte de notre travail pendant le stage. Il se décompose en six chapitres : Le premier chapitre fournit un aperçu des fonctionnalités prévues de la plate-forme ainsi qu'une présentation de l'architecture proposée de haut niveau. Le deuxième chapitre se penche sur les aspects de traitement des données du projet et comment on aborde les contraintes de performance qui étaient dans la solution précédente. Le troisième chapitre concerne la visualisation des données ; dans ce chapitre, nous verrons comment on a construit un tableau de bord d'analyse performant. Après avoir discuté de certains éléments du développement logiciel du projet, nous aborderons les pratiques DevOps qu'on suit tout au long de notre stage, et la façon dont nous les utilisons pour accélérer nos cycles de développement et de déploiement au chapitre quatre. Le cinquième chapitre explique comment nous intégrons la surveillance de l'infrastructure dans notre système, et la sixième partie aborde la sécurité, l'authentification, l'autorisation et la gestion des utilisateurs de la plate-forme.

Ce rapport sera achevé par une conclusion, qui établit le bilan du travail effectué.

Chapitre 1

Cadre général du projet

Introduction

Dans ce chapitre, nous présenterons le contexte général de notre projet. Nous commencerons par présenter l'organisation hôte et le contexte du projet, suivi d'une discussion du problème que nous essayons de résoudre et de la solution que nous proposons. Enfin, nous définirons notre processus de travail.

1.1 Présentation de l'organisme d'accueil

BUSINESS & AI est une société privée spécialisée en Intelligence Artificielle (IA) basée en Tunisie, France et Montréal et. Depuis sa création en 2020, BUSINESS & AI a connu une forte croissance à l'international plus qu'à l'échelle nationale/locale. La société opère actuellement dans différents domaines d'activité, visant à soutenir ses partenaires à anticiper les risques et leurs améliorer le processus de prise de décision.



FIG. 1.1 : Logo de la société

Mission et Vision

BUSINESS & AI a été fondée autour de deux principaux objectifs stratégiques :

- Construire un pont solide entre les dernières percées scientifiques et technologiques dans le domaine de l'IA et l'entreprise.

- Identification et résolution des défis les plus complexes confrontés par les entreprises en combinant l'intelligence artificielle (IA), la technologie et l'expertise commerciale.

Domaines d'activités

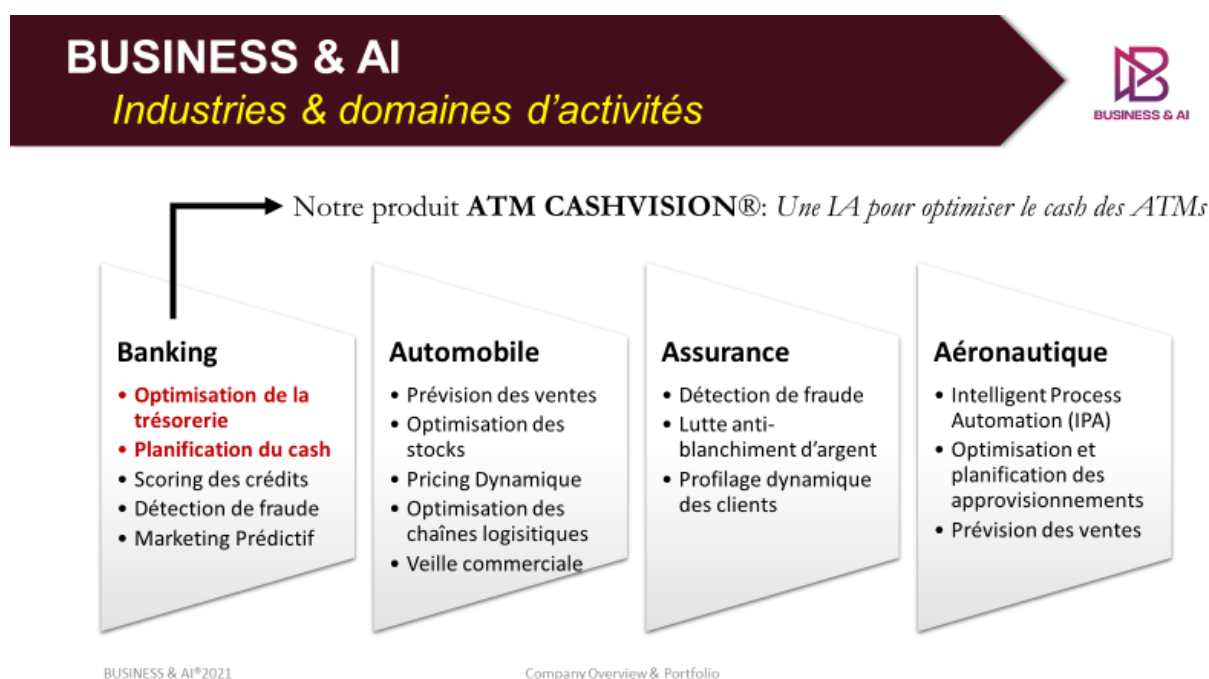


FIG. 1.2 : Domaines d'activités de la société

1.2 Cadre général du projet

Le présent projet intitulé **Cash Analytics Platform** a été élaboré dans le cadre de stage de fin d'étude en vue de l'obtention du diplôme national d'ingénieur informatique en Génie Logiciel-Architecture Logicielle à l'Institut Supérieur des Sciences Appliquées et de Technologie de Sousse pour l'année universitaire 2020/2021. Le travail est réalisé au sein de l'entreprise **Business & Ai**.

1.3 Étude de l'existant

La solution existante est une plateforme SaaS d'analyse des données relatives au fonctionnement des distributeurs automatiques de billets (DAB). Elle se base sur une technologie d'intelligence artificielle propriétaire de la société. Elle est complètement développée

en langage Python, en utilisant le framework Django, et elle suit une architecture monolithique modulaire. Ce n'est peut-être pas évident au début, mais le programme a de nombreuses limitations en raison de son architecture.

- Difficulté de mise en échelle (en Anglias : Scaling) : La mise en échelle d'une partie de l'application n'est pas possible. Toute l'application doit être déployée sur un serveur plus performant. Au cours du fonctionnement, la demande en ressources physiques varie d'un module à un autre : Il existe quelques modules, tels que les modules de traitement de données et celui qui fait appel aux modèles IA de prédictions, qui demandent une utilisation de mémoire et CPU importante. Le reste des modules sont relativement économes en ressources. Vu que les modules qui nécessitent des ressources importantes ne s'exécutent que pour des durées limitées de temps, la location de serveurs performant pour toute l'application est coûteux et n'est pas optimale.
- Difficulté de développement : Malgré que le langage Python est la norme pour le développement d'applications d'IA, son système de type dynamique pose un vrai problème dans le développement d'applications entreprise avec des bases de code énormes. Comme il n'y a pas de compilation statique ou de vérification de type, les erreurs ne sont détectées que lors de l'exécution de l'application.
- Difficulté d'intégration de collaborateurs : La charge cognitive requise pour comprendre toute la base du code augmente avec l'ajout et la modification des fonctionnalités.
- Cycle de développement lent : Une conséquence directe des deux derniers points est la lenteur de déploiement de nouvelles versions. À chaque fois toute l'application est déployée même si les changements n'affectent qu'un seul module du projet.

1.4 Solution proposée

Face aux difficultés mentionnées précédemment nous proposons une ré-architecture de la plateforme. En effet, il s'agit de trouver une stratégie de division des modules du système en micro-services et implémenter un prototype qui garantit la scalabilité horizontale et facilite la création de nouvelles fonctionnalités.

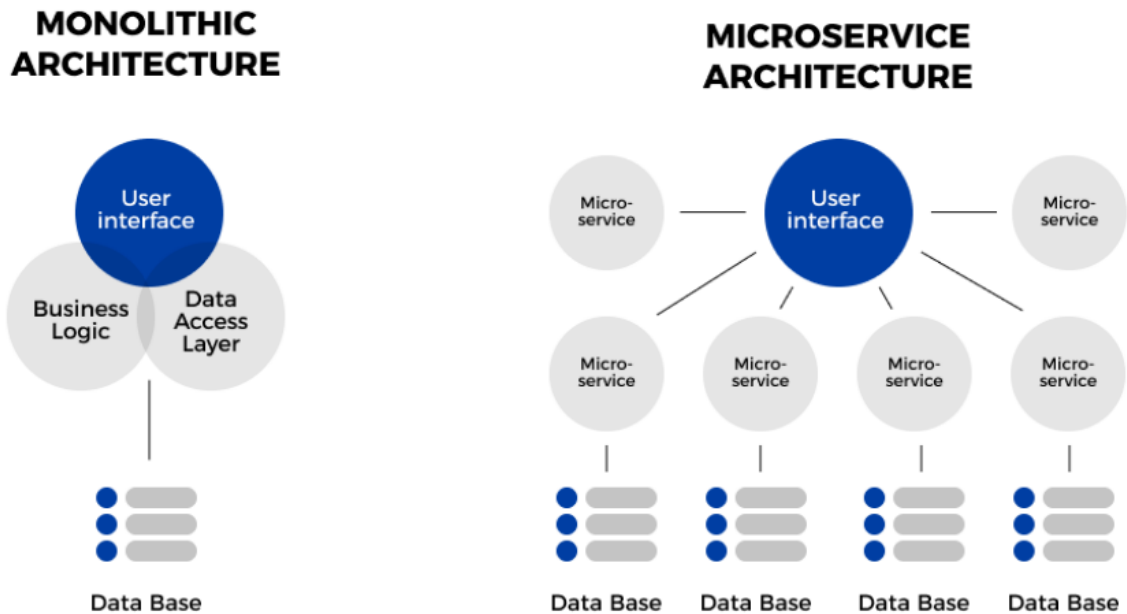


FIG. 1.3 : Différence entre les architectures [1]

La figure 1.3 montre la différence entre les deux architectures. Dans une architecture monolithique, tout le code de l'application est au même endroit et il est déployé sur un seul serveur. Contrairement à cela, une architecture micro-service divise une application en des petits sous-applications autonomes déployées séparément. Ces applications communiquent entre elles et avec le client pour offrir une expérience utilisateur cohérente et transparente.

1.4.1 Avantages de l'architecture micro-services

Les avantages des micro-services semblent suffisamment convaincants pour persuader plusieurs grandes entreprises, comme Amazon, Netflix et eBay, d'adopter le paradigme. Ce paradigme présente les avantages suivants par rapport à une architecture monolithique :

- **Optimisation d'utilisation des ressources** : Les modules du programme exigeants en ressources physiques peuvent être déployés sur du matériel plus puissant, tandis que les parties moins exigeantes peuvent être placées sur du matériel moins coûteux.
- **Choisir le meilleur outil pour chaque situation** : Les micro-services peuvent être développés dans différentes technologies tant qu'ils peuvent communiquer de manière standardisée. Cela permet à l'équipe de choisir différentes technologies pour

différentes exigences (Python pour l'IA, NodeJS pour le développement d'API, Go-lang pour les tâches de calcul intensif, etc.). Choisir le bon outil pour le travail donne une productivité plus élevée, un code plus facile à maintenir avec de meilleures performances.

- **Cycles de développement rapide et logiciels extensible** : Le développement peut se faire en parallèle avec des équipes plus petites. Les fonctionnalités peuvent être développées indépendamment et intégrées ultérieurement. Cela accélère la mise sur le marché d'un produit minimal, tout en ajoutant facilement plus de fonctionnalités plus tard.

1.4.2 Les défis de l'architecture des micro-services

Bien que cela puisse paraître simple à première vue, la création d'une architecture de micro-services n'est pas un processus anodin. En réalité, il offre plusieurs obstacles :

- La barrière à l'entrée est plus élevée que les applications monolithiques, nécessité des connaissances de l'administration et des opérations du système.
- Problèmes de systèmes distribués : une architecture de micro-services présente les mêmes défis d'un système distribué en ce qui concerne la surveillance et les vérifications de l'état, la journalisation distribuées de l'activité du système, l'équilibrage de charge et les performances globales du système, etc.
- Cela nécessite un investissement initial dans la recherche et la prise de décision concernant la division de la solution actuelle en applications plus petites, la fourniture d'une infrastructure et la mise en place d'un environnement de test et déploiement continue automatisée.

1.5 Objectifs

Notre objectif principal est de concevoir une architecture robuste qui nous permettrait de développer la plateforme d'analyse des données d'activité des DAB, étape par étape. De nombreux modules vont être reconstruits à partir de zéro et exposés en tant que services autonomes, d'autres vont être dérivés de la base de code existante. Pour notre stage, nous allons travailler sur :

- Refonte du pipeline de traitement des données.

- Conception et développement d'un tableau de bord analytique de visualisation des données.
- Conception et développement d'une couche de sécurité et contrôle d'accès aux services.
- Conception et développement d'un service de surveillance de l'infrastructure.

1.6 Méthodologie adoptée

Pour avoir une bonne gestion de projet, il est indispensable de choisir la méthodologie de travail appropriée à l'équipe, permettant de faire face aux contraintes temporelles et communicative. Ainsi dans cette section, nous présentons successivement les méthodes agiles, la méthodologie et les outils qu'on a choisis pour gérer le projet.

1.6.1 Les méthodologies agiles

Les méthodologies de gestion de projet sont inspiré du manifeste agile[2] édité en 2001 par un groupe d'experts du domaine de logiciels bien décidés à améliorer leur process et à réduire leur taux d'échec. Parmi les méthodes on note :

- Scrum[3]
- Kanban
- Extreme Programming (XP)[4]
- Rapid Application Development (RAD)[5]

1.6.2 Kanban

Kanban s'articule autour de quelque principes clés qui, une fois suivis, donneraient les meilleurs résultats.

1.6.3 Visualiser le flux de travail

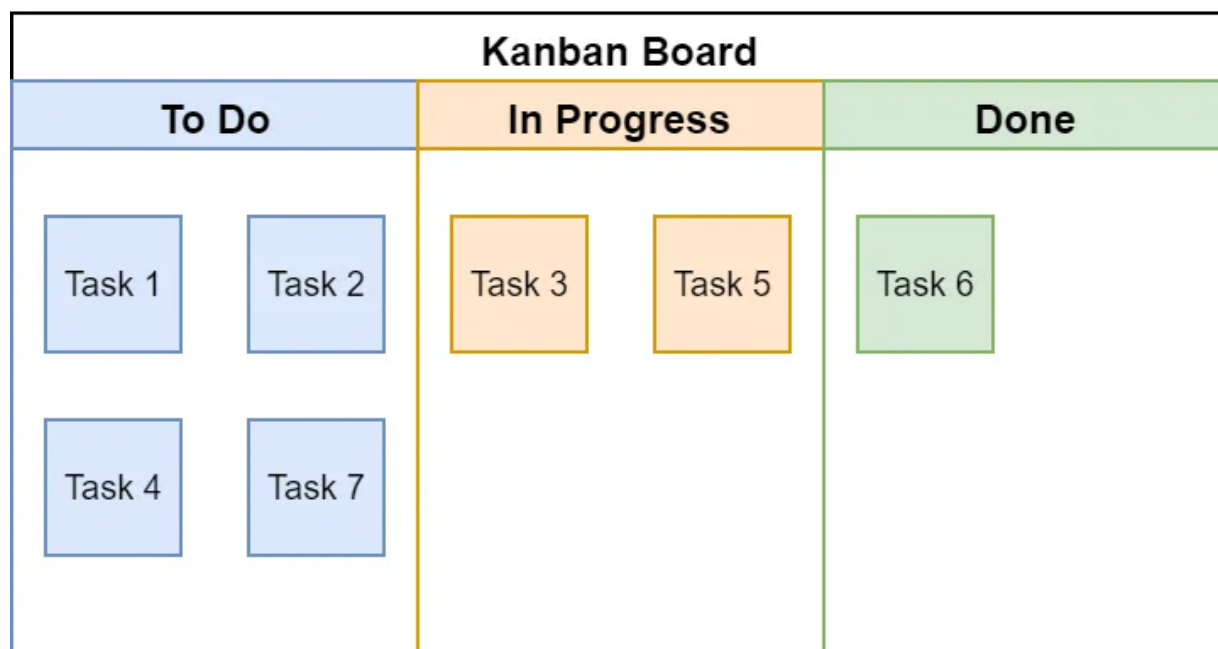


FIG. 1.4 : Un exemple de tableau Kanban

La visualisation du flux de travail est le premier des principes de cette méthode. Pour visualiser notre processus on utilise un tableau similaire à la figure 1.4. Chaque colonne du tableau représente une étape dans le flux de travail. Chaque carte Kanban représente un élément de travail. Le tableau Kanban lui-même reflète l'état actuel de notre processus, y compris tous ses risques et spécifications.

1.6.4 Limiter les travaux en cours

Kanban suggère l'adaptation de la charge de travail à la capacité de l'équipe. La limitation des tâches nécessite la mise en place d'un système tirage et la définition d'un nombre maximal de tâches par étape pour garantir qu'une carte n'est « tirée » dans l'étape suivante que lorsque la capacité le permet. Ces limitations mettront rapidement en évidence les zones problématiques dans un flux de travail pour que nous puissions y réfléchir et les corriger.

1.6.5 Processus explicites

Le processus doit être bien spécifié. Avoir une "Definition Of Done" pour chaque phase du processus signifie qu'un élément doit répondre à des critères clairs avant de pouvoir être transférer a l'étape suivante.

1.6.6 Livraison continue

Kanban encourage le déploiement continu de petites fonctionnalités ou corrections de bugs. Cette méthode favorise les pratiques DevOps pour augmenter la productivité.

Conclusion

Dans ce chapitre, nous avons présenté notre organisation d'accueil, Business-and-Ai, ainsi que le problème de leur application monolithique et les objectifs que nous espérons atteindre au cours de notre stage.

Chapitre 2

Spécifications et architecture globale

Introduction

Dans ce chapitre, nous définirons les exigences fonctionnelles et non fonctionnelles du système, ainsi que ses cas d'utilisation généraux. Nous offrirons un aperçu de haut niveau de la conception globale du système, suivi d'une discussion sur les décisions techniques que nous avons prises pour le faire fonctionner.

2.1 Spécification des besoins

Dans cette partie on va introduire les acteurs du système, les fonctionnalités attendues ainsi que les besoins non fonctionnels que nos utilisateurs anticipent.

2.2 Les acteurs

L'une des premières étapes de la conception d'un système consiste à identifier les acteurs de notre future application. Un acteur est une entité externe au système. Il représente une personne ou un autre système informatique qui attend un ou plusieurs services offerts par une interface d'accès. Notre application admet deux acteurs :

- **Analyste** : C'est la personne responsable de l'analyse et interprétation des données issues de l'activité des DABs, ainsi qu'écrire des rapports décrivant les rendements d'un groupe ou d'un seul DAB.
- **Administrateur de système** : Un administrateur système possède tous les privilèges et droit d'accès au système. C'est la personne responsable de la gestion de l'application, son infrastructure et ses utilisateurs.

2.3 Besoins fonctionnels

Les besoins fonctionnels sont des fonctionnalités qui répondent aux besoins des utilisateurs. Cette plate-forme doit couvrir principalement les besoins fonctionnels suivants :

- Traiter automatiquement les données soumises par l'utilisateur.
- Consulter le tableau de bord analytique
- Gérer les utilisateurs et les rôles du système
- Surveiller l'infrastructure du système

2.4 Besoins non fonctionnels

Sont des caractéristiques qui devraient être présentes pour que notre système soit considéré compétent et adéquat pour les tâches pour lesquelles il a été conçu

- **Sécurité du système** : La plate-forme doit être conforme aux standards de la sécurité des applications web modernes. Tous les échanges de données doivent être chiffrés, les accès aux différentes parties du système doivent être authentifiés.
- **Performance** : Le temps de réponse des requêtes au système doit être minimale quel que soit le type et charge de travail effectué.
- **Tolérance aux pannes** : Suite à une panne imprévisible, le système doit être facile à rétablir et le problème facile à identifier
- **Adaptatifs aux types de clients** : Toute interface doit s'adapter aux écrans mobiles, tablettes ou Desktop tout en gardant une expérience utilisateur agréable.
- **Évolutivité** : L'ajout de nouvelles fonctionnalités doit être assez facile et le couplage entre les parties du système doit être minimale. Chaque service du système peut être déployé séparément.

2.5 Diagrammes de cas d'utilisation

2.5.1 Cas d'utilisations globaux

Le diagramme suivant est une représentation simplifiée des fonctionnalités attendues du système selon le type d'utilisateur

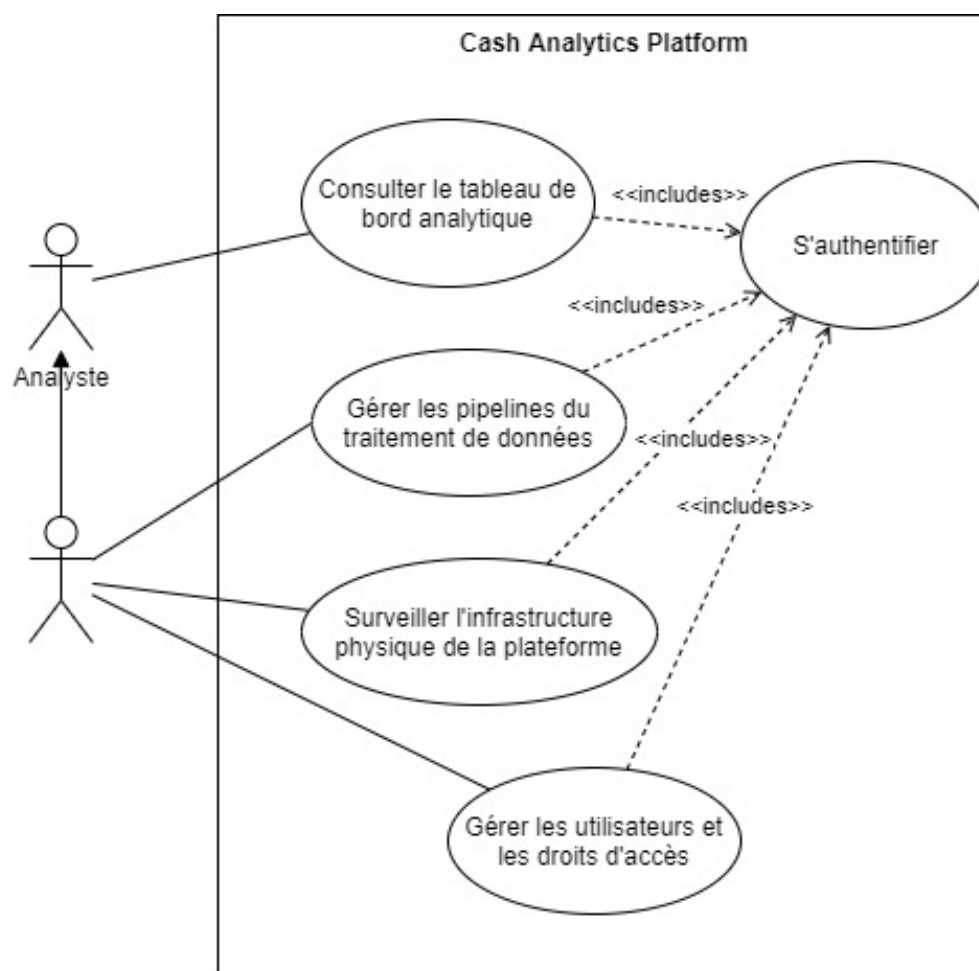


FIG. 2.1 : Diagramme de cas d'utilisation global

Chacun de ces cas d'utilisation est composé de plusieurs sous-fonctionnalités.

2.5.2 Tableau de bord analytique

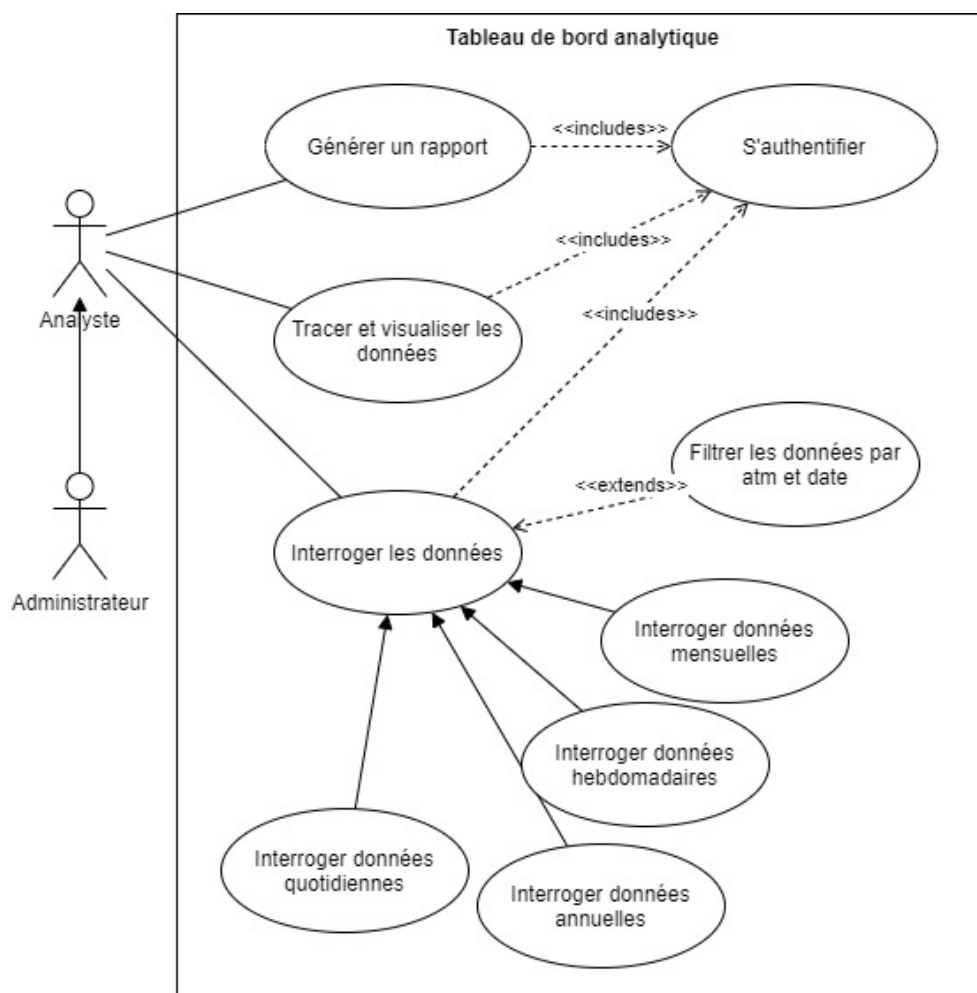


FIG. 2.2 : Cas d'utilisation du tableau de bord analytique

La figure 2.2 décrit les cas d'utilisation du tableau de bord analytique. C'est la partie de l'application qui permet d'explorer et les données des DABs. La plupart des informations dont nous disposons sur les distributeurs automatiques sont indexées dans le temps, le tableau de bord analytique permet à l'utilisateur de rechercher des informations dans des plages de dates spécifiques et de visualiser les données par jours, semaine, mois ou année.

2.5.3 Gestion des pipelines de traitement des données

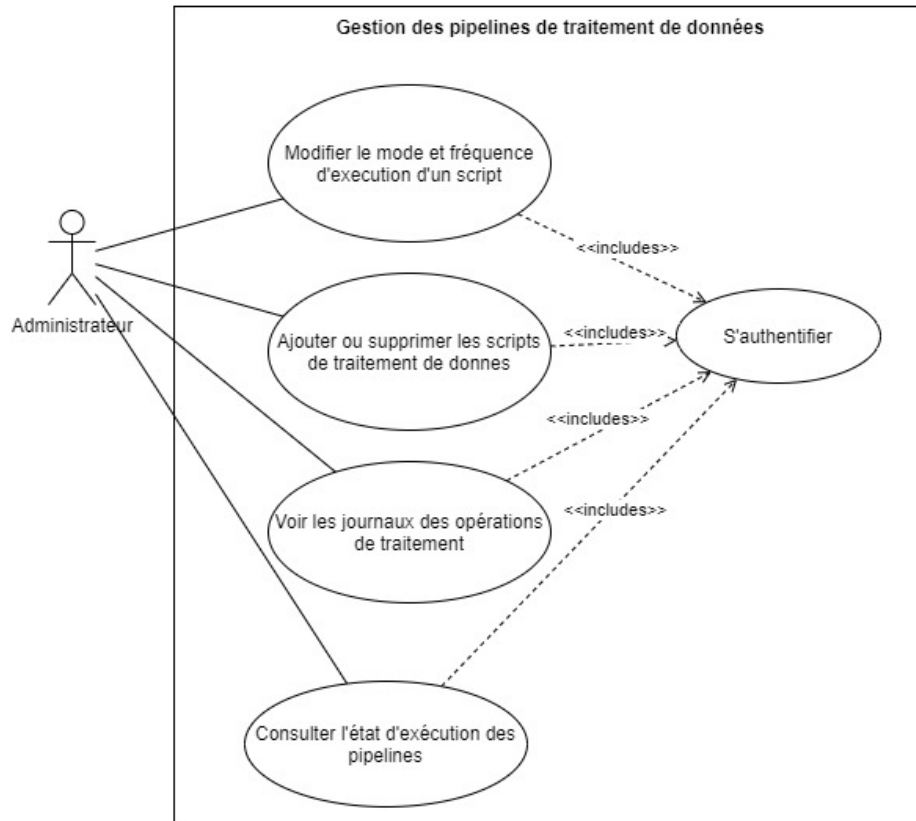


FIG. 2.3 : Diagramme CU de la gestion des pipelines de traitement des données

La figure 2.3 illustre l'interaction de l'administrateur avec le système de gestion des données. Il est capable de gérer les scripts de traitement, ainsi que contrôler le moment et la fréquence d'exécution de ces fichiers. L'administrateur peut aussi voir l'état d'exécution et les journaux d'opérations.

2.6 Surveillance d'infrastructure

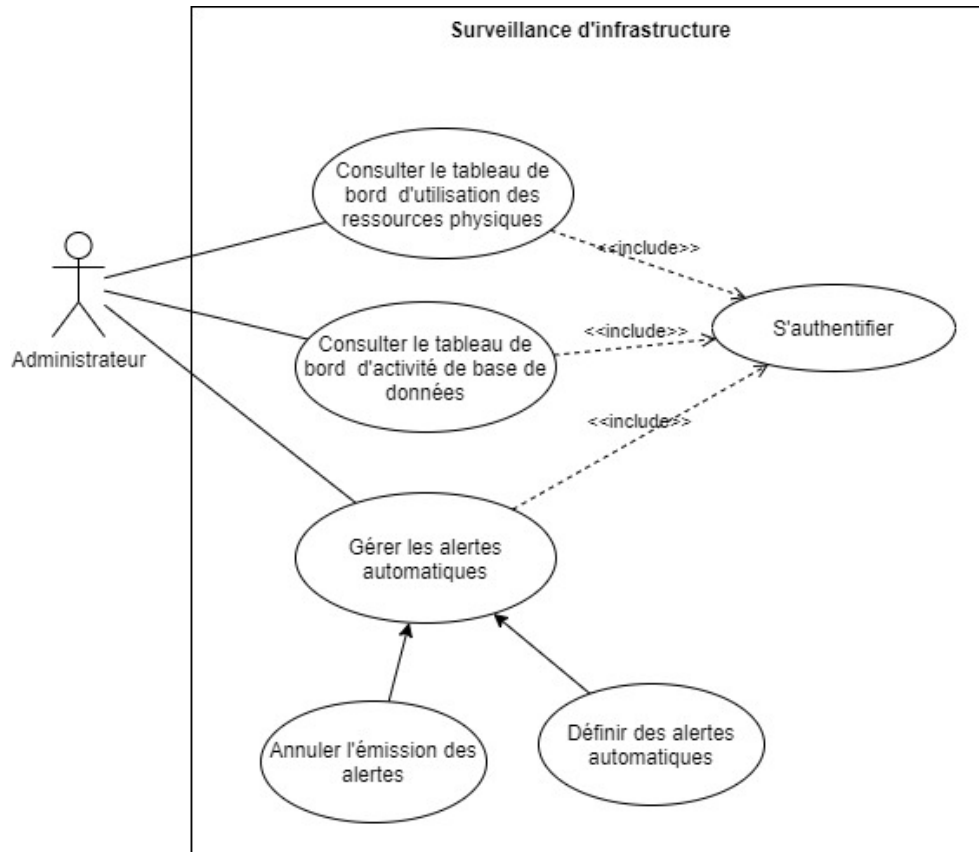


FIG. 2.4 : Diagramme CU de la surveillance d'infrastructure

Le comportement du système et l'utilisation des ressources donnent des informations importantes sur la qualité du logiciel que nous avons déployé. La figure 2.4 illustre les cas d'utilisation du système de surveillance de l'infrastructure ; l'administrateur système doit être capable de visualiser l'utilisation des ressources physique pour chaque micro-service ainsi que l'utilisation de la base de données (sessions actives, tuples ajoutés, tuple cherchés, etc.).

2.7 Architecture globale du système

Après avoir eu une idée sur les fonctionnalités de notre application, nous allons discuter l'architecture logique. Notre objectif est de diviser ces fonctionnalités en micro-services autonomes avec un couplage minimal.

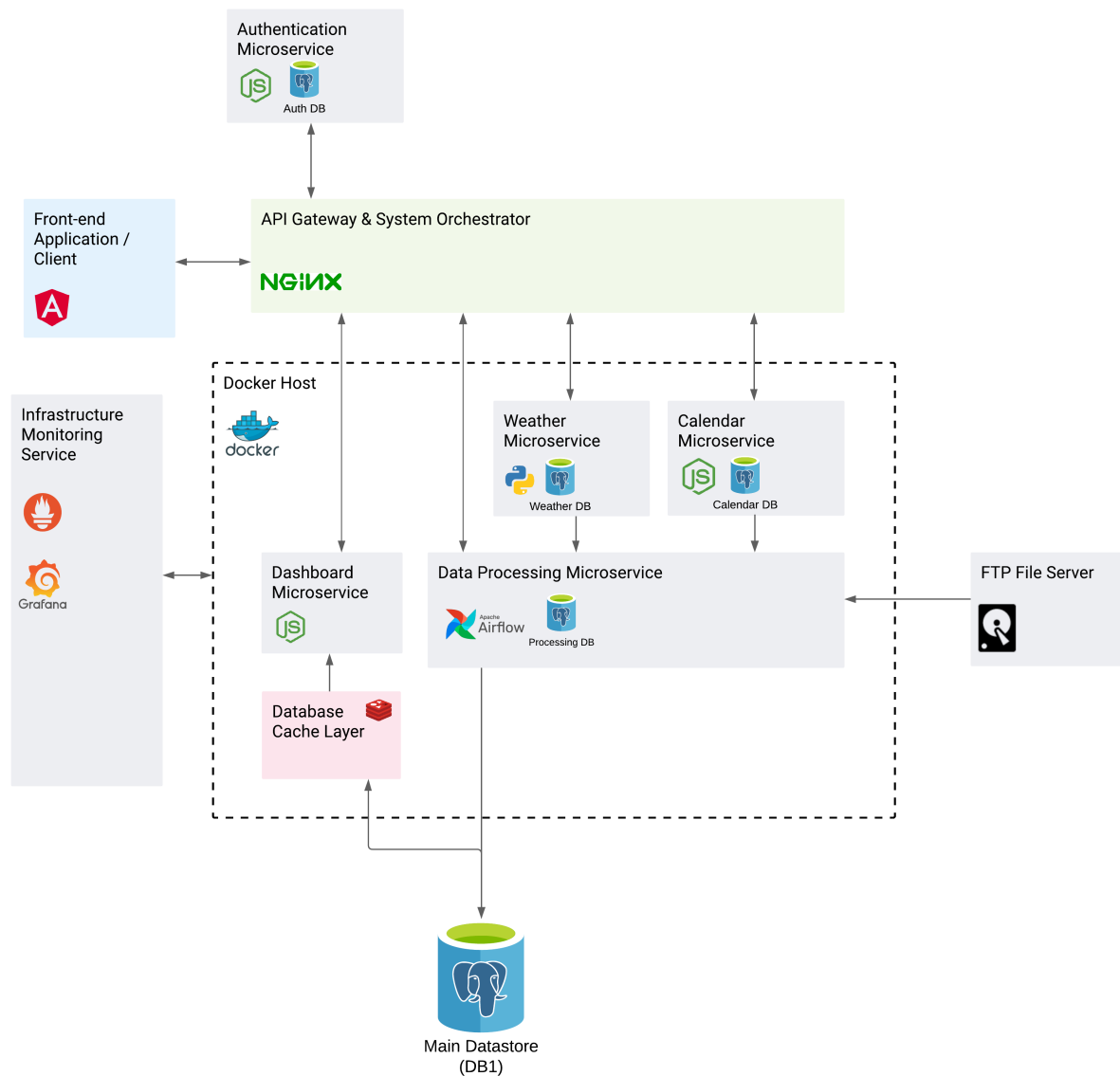


FIG. 2.5 : Architecture logique globale

La figure 2.5 est une représentation de l'architecture logique de la plateforme. Elle se compose de :

- **API Gateway** : C'est une passerelle qui gère tous les accès aux services backend. Toutes les requêtes sont acheminées via ce serveur pour authentifier, à l'aide du micro-service de sécurité.
- **Client Application** : L'application client permet à l'utilisateur d'interroger les services backend de manière efficace et consolidée.

- **Dashboard Microservice** : Un micr-oservice très simple qui interroge la base de données centrale pour le compte de l'application cliente. Il fournit une interface qui renvoie les données de manière structurée et significative au client. Les requêtes à la base de données sont mises en cache pour améliorer les performances des appels suivants.
- **Authentication Microservice** : Chaque requête transmise par la passerelle API aux services backend est d'abord acheminée vers le service d'authentification, qui détermine si l'expéditeur est autorisé à accéder au service.
- **Weather and Calendar Micro-services** : Ces deux micro-services sont utilisés par le service de traitement des données pour enrichir les données brutes (ajouter plus d'informations sur la météo et les événements (vacances) survenues dans une période donnée). Chacun de ces services possède sa propre base de données et fournit une interface API REST qui peut être utilisée en externe par d'autres développeurs.
- **Data Processing Service** : C'est un ensemble d'applications en charge de la planification et de l'exécution des scripts de traitement des données. Ce groupe de services peut accéder au serveur FTP, où le client est censé envoyer les fichiers de travail pour enrichir et nettoyer les données avant de les soumettre dans une base de données centrale. De plus, il fournit une interface pour la gestion des scripts.
- **Infrastructure Monitoring Service** : Le service de surveillance de l'infrastructure est un ensemble d'applications qui servent essentiellement à surveiller la consommation des ressources physiques. Dans les prochains chapitres, nous verrons plus en détail comment tout fonctionne ensemble.

2.8 La Virtualisation d'environnement d'exécution

L'architecture qu'on a introduit dans la section précédente s'appuie fortement la virtualisation d'environnement d'exécution pour le déploiement de nos services. Nous avons choisi d'utiliser Docker pour notre plateforme de conteneurisation.

2.8.1 Introduction à Docker

Docker est une plate-forme qui permet la création, exécuter et le déploiement des applications sous forme de conteneurs isolés sans tenir compte de la plateforme ou le système d'exploitation de la machine hôte.

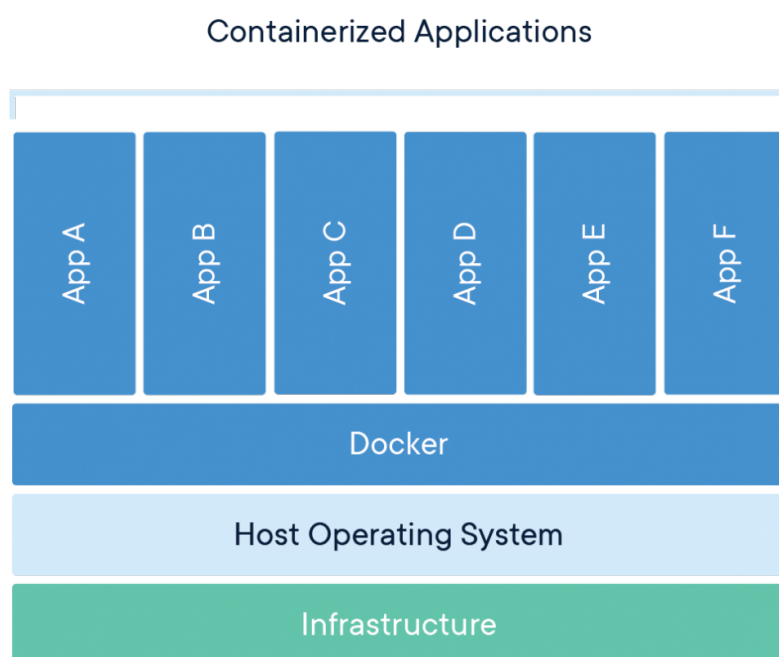


FIG. 2.6 : Déploiement de conteneurs logiciels

Un conteneur est une unité logicielle standard qui regroupe le code et toutes ses dépendances, de sorte que l'application s'exécute rapidement et de manière fiable. Les conteneurs sont légers et peuvent être facilement répliqués, mis à jour ou détruits. Ils sont créés en instanciant une image docker qui comprend tout ce qu'on a besoin pour exécuter une application tel que le code, environnement d'exécution, les programmes ou autres dépendances.

La figure 2.6 illustre le déploiement de conteneurs sur une machine. Le service Docker installé sur le système d'exploitation de l'hôte est responsable de l'exécution et de la gestion des conteneurs.

Tous nos micro-services sont conteneurisés et les images sont mises à jour régulièrement à chaque mise à jour du code grâce aux pipelines CI/CD (plus d'informations dans les prochains chapitres).

Conclusion

Dans ce chapitre, nous avons discuté les critères fonctionnels et non fonctionnels. Nous avons fourni un aperçu de haut niveau de notre conception proposée en mettant en évidence ses composants essentiels.

Chapitre 3

Récupération et traitement des données

Introduction

Dans ce chapitre, nous commencerons par introduire les données et le processus de nettoyage. Ensuite nous présenterons nos choix techniques pour cette partie de la plateforme. Enfin nous montrons quelques captures d'écrans de l'implémentation.

3.1 Description du processus

Pour simplifier la procédure, nous prenons les fichiers CSV du client, exécutons du code dessus, puis enregistrons la sortie dans une base de données. Un fichier "Comma Separated Values" (CSV) est un simple fichier texte qui contient des données sous format tabulaire.

3.1.1 Les données des clients

La procédure de traitement des données commence par l'envoi par le client de trois fichiers CSV contenant des informations sur les activités quotidiennes d'un ensemble de distributeurs automatiques de billets :

- **withdrawals.csv** : Ce fichier contient une liste des opérations de retrait pour un groupe de guichets automatiques. Chaque opération de retrait comprend le montant, le code de référence du guichet, l'horodatage de l'opération, si elle a été effectuée par un client et les éventuels frais supplémentaires.
- **cash-load.csv** : On appelle l'opération de dépôt d'espèces dans un guichet automatique "cash-load". L'agent de maintenance en charge de ce guichet effectue cette tâche manuellement. Un historique des activités de chargement est enregistré dans cash-load.csv. Chaque ligne contient une référence au guichet automatique, le montant, la devise et l'horodatage de l'opération.

- **cash-unload.csv** : Le format de ce fichier est assez similaire à celui de cash-load.csv ; il contient les mêmes colonnes, mais elles décrivent le processus de déchargement d'un guichet automatique.

3.1.2 Étapes du traitement des données

Le traitement des données est un processus incrémentiel à plusieurs phases dans lequel chaque étape s'appuie sur la précédente :

- **Récupération de données et validation de format** : Les données obtenues d'un client sur internet ne doivent jamais être dignes de confiance. La première étape du processus de nettoyage consiste à vérifier la structure de chaque fichier ; Les fichiers doivent respecter les règles de format CSV et inclure les colonnes de données appropriées.
- **Nettoyage des données incohérentes** : Cette étape est très importante pour cash-load.csv et cash-unload.csv car ces fichiers contiennent des valeurs erronées ou manquantes en raison d'une erreur humaine. Étant donné qu'il s'agit de données de séries chronologiques de nature périodique, ces erreurs peuvent être reconnues et corrigées par programmation.
- **Reconstitution** : Ces données brutes sont peu pratiques à utiliser dans le cadre d'analyse de données. Le plus souvent, des données supplémentaires, telles que des données de calendrier ou météorologiques, sont utilisées en conjonction avec les données réelles, pour obtenir et clarifier le contexte des informations des guichets automatiques. Il est également possible de pré-calculer certaines valeurs ou faire des agrégations sur des colonnes existantes pour augmenter les performances des requêtes.
- **Application des modèles d'IA à l'enrichissement des données** Enfin, l'intégration de la technologie d'IA propriétaire à la société. C'est la phase finale, et c'est là que se trouve la vraie valeur ajoutée de la solution.

3.2 Étude et spécifications

La nouvelle solution doit répondre aux problèmes de l'application existante :

- **Extensibilité** : Dans l'architecture monolithique, les modifications simples dans les scripts de traitement de données nécessiteraient un redéploiement complet de la plateforme. Le développement des modèles d'IA est un processus itératif et progressif qui nécessite beaucoup d'ajustements et d'essais, ce qui rend la solution actuelle moins productive.
- **Intégration avec des systèmes externes** : L'intégration avec les outils existants et les plateformes cloud nécessite une intervention manuelle ou l'utilisation de bibliothèques tierces. Le changement de fournisseur de services cloud, par exemple, nécessiterait de profonds changements dans la base de code.
- **Journalisation et gestion des erreurs** : La journalisation et la gestion des erreurs font partie du code qui effectue le traitement ; ce n'est pas idéal, car cela rend le code moins lisible, moins réutilisable.

3.2.1 Solutions Existantes

Vu que le problème du traitement des fichiers texte massifs est particulièrement répandu dans l'analyse de données, nous avons commencé à étudier et à tester de nombreux outils et frameworks qui se sont avérés utiles dans l'industrie :

	Spotify Luigi	Apache Airflow	Apache Oozie
Description	Spotify Luigi[6] est un module Python qui permet la création de pipelines de tâches complexes. Il prend en charge la résolution des dépendances, la gestion des flux de travail, la visualisation, la gestion des défaillances et la communication avec les systèmes externes.	Apache Airflow[7] est une plate-forme de gestion de flux de travail. Il dispose d'un ensemble complet de fonctionnalités pour la création de processus, la gestion des erreurs et la surveillance de l'exécution. Il s'intègre facilement avec les outils de big data (Hadoop, Spark, Hive) et les services cloud populaires (AWS, Azure, GCP).	Apache Oozie[8] est une solution de planification de workflow pour la gestion des tâches Hadoop. Les workflows dans Oozie sont définis comme une collection de flux de contrôle et de nœuds d'action dans un graphe acyclique dirigé.
Philosophie	Un processus est représenté sous la forme d'un graphique de dépendance des tâches qui peuvent être de simples scripts python ou autres processus.	Les flux de travail sont représentés sous forme de graphes dirigés acycliques de tâches ou de scripts exécutables.	Les flux de travail sont représentés sous forme de graphes dirigés acycliques de tâches ou de scripts exécutables.
Language	Python	Python	XML, Java
GitHub starts	14K	22K	615
Première sortie	09-2012	06-2015	06-2012

TAB. 3.1 : Comparaison des outils de gestion de flux de travail open source populaires

Le tableau 3.1 résume une étude comparative des outils de workflow existants. Bien que populaire parmi les analystes de données, Apache Oozie ne nous convient pas, car il est étroitement couplé aux tâches Hadoop et utilise XML comme langage de spécification de flux de travail, ce qui rend la transition vers cette technologie plus difficile.

Après avoir testé ces frameworks, nous avons décidé de choisir Apache Airflow pour créer notre pipeline de traitement de données pour les raisons suivantes :

- Plus adopté par l'industrie, Airflow est le plus populaire des trois, bien qu'il soit le plus récente.
- Le paradigme du graphe acyclique dirigé est intuitif et facile à utiliser par rapport au modèle de dépendance des tâches de Luigi.
- Un meilleur soutien communautaire. Luigi n'a pas encore de documentation ou d'image docker officielle.

3.2.2 Introduction à Airflow

Airflow est une plateforme qui permet de créer et de gérer des processus. Un workflow est représenté sous la forme d'un DAG (Graphique acyclique dirigé) et se compose de morceaux de travail distincts (tâches) qui sont organisés en tenant compte des dépendances et des flux de données.

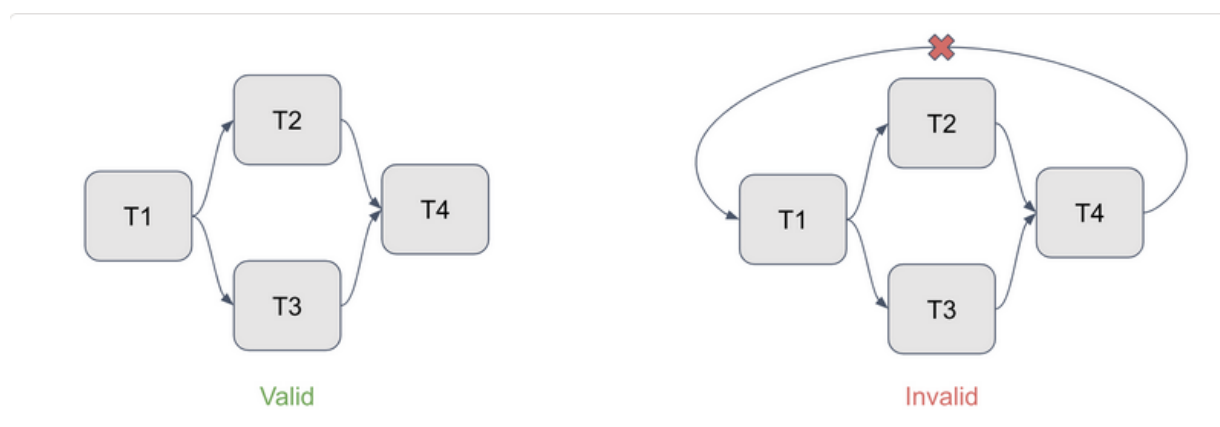


FIG. 3.1 : Exemple d'un DAG

Un DAG décrit les dépendances entre les tâches, ainsi que la séquence dans laquelle elles doivent être exécutées; les tâches elles-mêmes spécifient ce qu'il faut faire, qu'il s'agisse d'obtenir des données, d'effectuer des analyses, d'activer d'autres systèmes ou toute autre chose.

3.2.3 Architecture

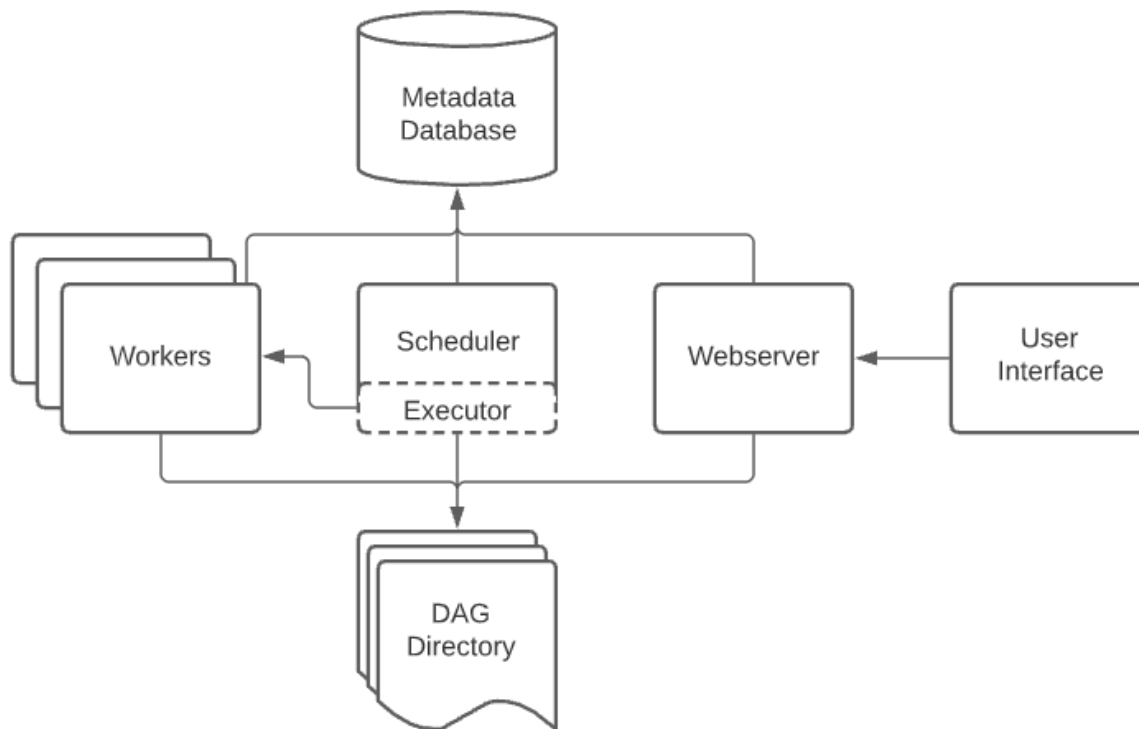


FIG. 3.2 : Architecture logique de Airflow

La figure 3.2 détaille les différents composants qui constituent Airflow :

- **Scheduler** : C'est le planificateur des tâches ; il est chargé d'envoyer les processus à l'exécuteur quand c'est le temps.
- **Executor** : Un exécuteur est un programme qui gère les tâches en cours d'exécution. Il communique avec le planificateur pour exécuter les travaux. Dans un environnement de production, de nombreux exécuteurs peuvent être déployés sur différents serveurs pour augmenter la performance.
- **DAGs folder** : Un dossier géré par le planificateur. Contient les scripts qui contiennent le code des DAG.
- **Web Server** : Un serveur Web qui fournit une interface utilisateur pour inspecter, déclencher et déboguer les DAG.

3.2.4 Résolution des problèmes liés à l'utilisation de fichiers CSV

Travailler avec des fichiers directement en mémoire est l'une des principales contraintes de performances de la solution monolithique. Les fichiers CSV ont tendance à être volumineux et sont lus de manière séquentielle ; le filtrage ou la recherche de valeurs nécessite de parcourir les lignes une par une. Nous avons choisi d'intégrer une base de données de traitement temporaire qui stockera plutôt les données contenues dans les fichiers CSV dans des tables.

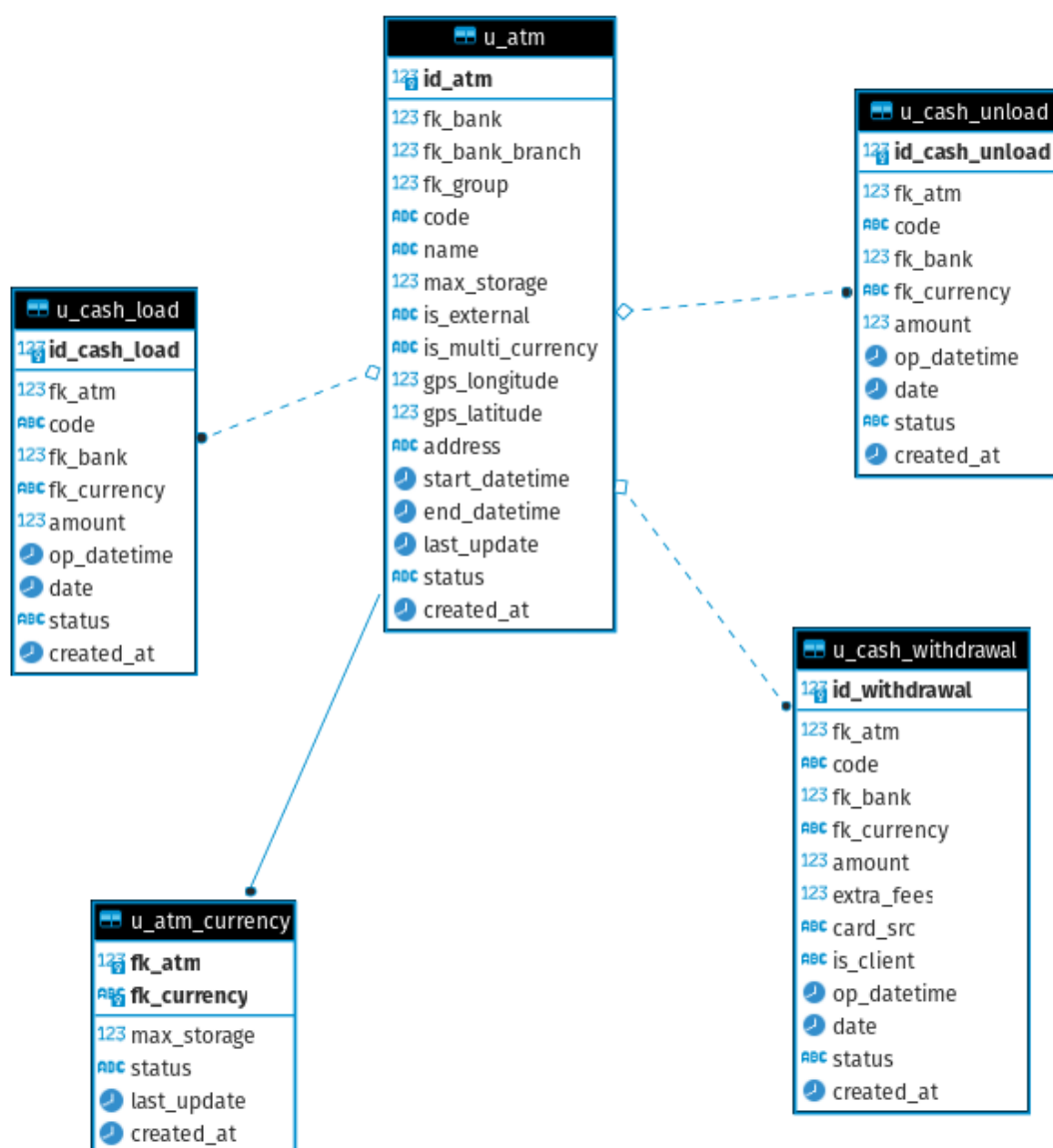


FIG. 3.3 : Schéma de la base de données des traitements

3.3 Implémentation

DAG	Runs	Schedule	Last Run
☐ ai_continuous_daily_predictions	16 16	@daily	2021-06-25, 00:00:00
☐ ai_features_engineering_dag	13 16	@daily	2021-06-25, 00:00:00
☐ calendar_data_pulling_dag	1	@once	2021-06-27, 18:38:35
☐ dag_transform_tsfe	2	1 day, 0:00:00	2021-06-26, 18:38:49
☒ daily_cash_level_calculation_dag daily	387	@once	2021-07-01, 12:01:06
☒ daily_checking_dag daily verification	379	None	2021-07-01, 12:00:17
☒ daily_data_construction_dag daily	374	None	2021-07-01, 12:00:36
☒ daily_migration_db0_db1_dag daily migration	374	None	2021-07-01, 12:00:28

FIG. 3.4 : Liste des DAGs

FIG. 3.5 : Le dossier contenant les DAGs

Airflow détecte automatiquement les changements des fichiers dans le dossier des DAGs de la figure 3.5 et déploie les changements automatiquement.

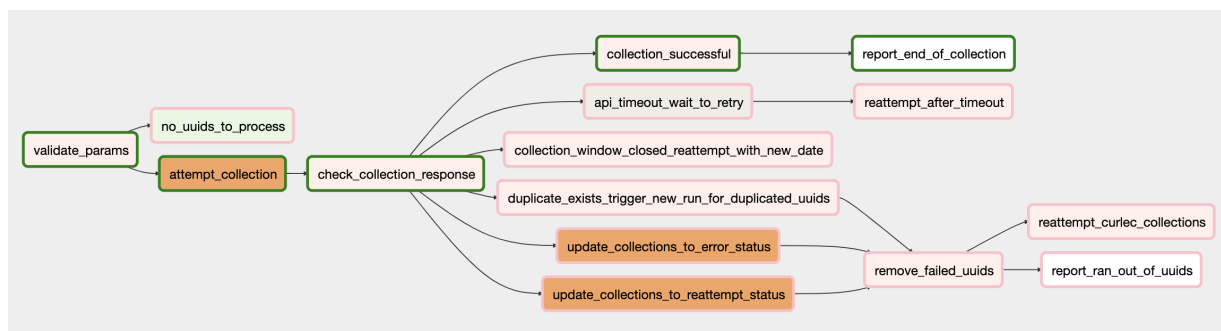


FIG. 3.6 : Exemple d'un DAG

La figure 3.6 représente une partie du DAG qui ajoute des données de calendrier (le fêtes et les vacances) aux données des DABs. Les tâches avec bordure rose représentent des actions alternatives qui doivent prendre effet dans des scénarios d'erreur (API call time- out, données dupliquées, plage de dates invalide, etc.). Ces tâches de gestion d'erreurs sont développées séparément et peuvent être réutilisées dans d'autres DAGs.

Task Instance: features_C5_atm7 at 2021-06-12, 02:00:00

Task Instance Details <> Rendered Template Log XCom

Log by attempts

1

Jump To End Toggle Wrap

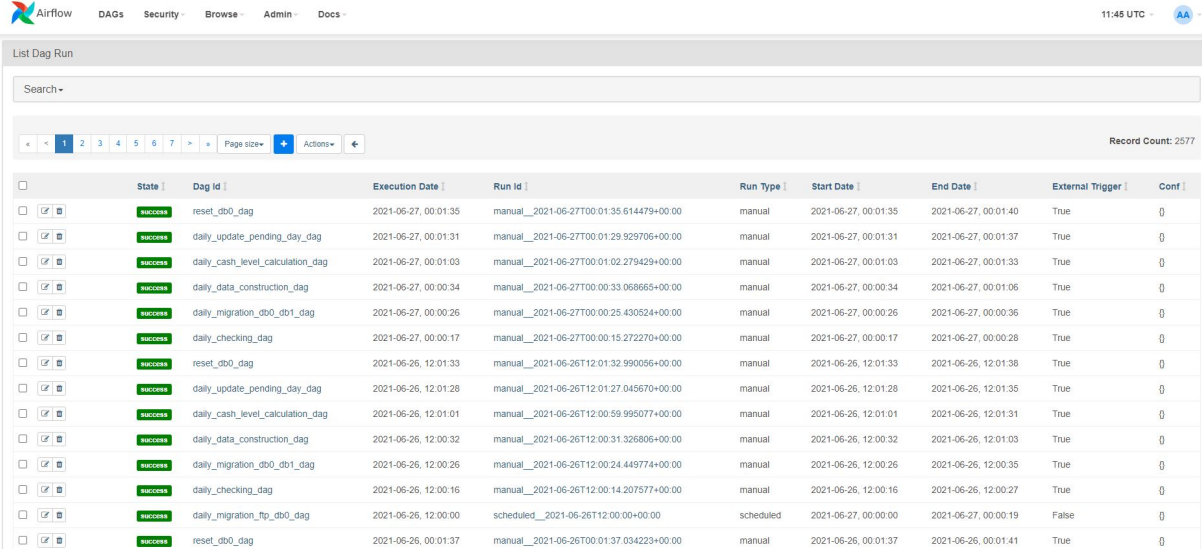
```

*** Reading local file: /opt/airflow/logs/ai_features_engineering_dag/features_C5_atm7/2021-06-12T00:00:00+00:00/1.log
[2021-06-27 19:54:52,738] {taskinstance.py:876} INFO - Dependencies all met for <TaskInstance: ai_features_engineering_dag.features_C5_atm7 2021-06-12T00:00:00+00:00 [queued]>
[2021-06-27 19:54:52,767] {taskinstance.py:876} INFO - Dependencies all met for <TaskInstance: ai_features_engineering_dag.features_C5_atm7 2021-06-12T00:00:00+00:00 [queued]>
[2021-06-27 19:54:52,767] {taskinstance.py:1067} INFO -
-----
[2021-06-27 19:54:52,767] {taskinstance.py:1068} INFO - Starting attempt 1 of 2
[2021-06-27 19:54:52,767] {taskinstance.py:1069} INFO -
-----
[2021-06-27 19:54:52,811] {taskinstance.py:1087} INFO - Executing <Task(PythonOperator): features_C5_atm7> on 2021-06-12T00:00:00+00:00
[2021-06-27 19:54:52,840] {standard_task_runner.py:76} INFO - Running: ['***', 'tasks', 'run', 'ai_features_engineering_dag', 'features_C5_atm7', '2021-06-12T00:00:00+00:00', '--job-id', '22924', '--pool', 'default_pool', '--raw', '--su
[2021-06-27 19:54:52,840] {standard_task_runner.py:77} INFO - Job 22924: Subtask features_C5_atm7
[2021-06-27 19:54:52,847] {standard_task_runner.py:52} INFO - Started process 9698 to run task
[2021-06-27 19:54:52,948] {logging_mixin.py:104} INFO - Running <TaskInstance: ai_features_engineering_dag.features_C5_atm7 2021-06-12T00:00:00+00:00 [running]> on host e47d5884a5f9
[2021-06-27 19:54:53,056] {taskinstance.py:1282} INFO - Exporting the following env vars:
AIRFLOW_CTX_DAG_OWNER=Souhi1
AIRFLOW_CTX_DAG_ID=ai_features_engineering_dag
AIRFLOW_CTX_TASK_ID=features_C5_atm7
AIRFLOW_CTX_EXECUTION_DATE=2021-06-12T00:00:00+00:00
AIRFLOW_CTX_DAG_RUN_ID=scheduled_2021-06-12T00:00:00+00:00
[2021-06-27 19:54:53,071] {base.py:78} INFO - Using connection to id: postgres_connection. Host: 51.77.221.56, Port: 30054, Schema: , Login: atmashy_user, Password: ***, extra: {}
[2021-06-27 19:54:54,073] {logging_mixin.py:104} WARNING - /opt/***/dags/continuous_api/secondary_model_builder.py:51 SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
[2021-06-27 19:54:54,622] {python.py:151} INFO - Done. Returned value was: None
[2021-06-27 19:54:54,644] {taskinstance.py:1191} INFO - Marking task as SUCCESS. dag_id=ai_features_engineering_dag, task_id=features_C5_atm7, execution_date=20210612T000000, start_date=20210627T195452, end_date=20210627T195454

```

FIG. 3.7 : Journal d'exécution d'une tâche



	State	Dag Id	Execution Date	Run Id	Run Type	Start Date	End Date	External Trigger	Conf
☐	success	reset_db0_dag	2021-06-27, 00:01:35	manual__2021-06-27T00:01:35.614479+00:00	manual	2021-06-27, 00:01:35	2021-06-27, 00:01:40	True	0
☐	success	daily_update_pending_day_dag	2021-06-27, 00:01:31	manual__2021-06-27T00:01:29.929706+00:00	manual	2021-06-27, 00:01:31	2021-06-27, 00:01:37	True	0
☐	success	daily_cash_level_calculation_dag	2021-06-27, 00:01:03	manual__2021-06-27T00:01:02.279429+00:00	manual	2021-06-27, 00:01:03	2021-06-27, 00:01:33	True	0
☐	success	daily_data_construction_dag	2021-06-27, 00:00:34	manual__2021-06-27T00:00:33.068665+00:00	manual	2021-06-27, 00:00:34	2021-06-27, 00:01:06	True	0
☐	success	daily_migration_db0_db1_dag	2021-06-27, 00:00:26	manual__2021-06-27T00:00:25.430524+00:00	manual	2021-06-27, 00:00:26	2021-06-27, 00:00:36	True	0
☐	success	daily_checking_dag	2021-06-27, 00:00:17	manual__2021-06-27T00:00:15.272270+00:00	manual	2021-06-27, 00:00:17	2021-06-27, 00:00:28	True	0
☐	success	reset_db0_dag	2021-06-26, 12:01:33	manual__2021-06-26T12:01:32.990056+00:00	manual	2021-06-26, 12:01:33	2021-06-26, 12:01:38	True	0
☐	success	daily_update_pending_day_dag	2021-06-26, 12:01:28	manual__2021-06-26T12:01:27.045670+00:00	manual	2021-06-26, 12:01:28	2021-06-26, 12:01:35	True	0
☐	success	daily_cash_level_calculation_dag	2021-06-26, 12:01:01	manual__2021-06-26T12:00:59.995077+00:00	manual	2021-06-26, 12:01:01	2021-06-26, 12:01:31	True	0
☐	success	daily_data_construction_dag	2021-06-26, 12:00:32	manual__2021-06-26T12:00:31.326806+00:00	manual	2021-06-26, 12:00:32	2021-06-26, 12:01:03	True	0
☐	success	daily_migration_db0_db1_dag	2021-06-26, 12:00:26	manual__2021-06-26T12:00:24.449774+00:00	manual	2021-06-26, 12:00:26	2021-06-26, 12:00:35	True	0
☐	success	daily_checking_dag	2021-06-26, 12:00:16	manual__2021-06-26T12:00:14.207577+00:00	manual	2021-06-26, 12:00:16	2021-06-26, 12:00:27	True	0
☐	success	daily_migration_ftp_db0_dag	2021-06-26, 12:00:00	scheduled__2021-06-26T12:00:00+00:00	scheduled	2021-06-27, 00:00:00	2021-06-27, 00:00:19	False	0
☐	success	reset_db0_dag	2021-06-26, 00:01:37	manual__2021-06-26T00:01:37.034223+00:00	manual	2021-06-26, 00:01:37	2021-06-26, 00:01:41	True	0

FIG. 3.8 : Journal d'exécution d'une tâche

les Figures 3.7 et 3.8 représentent les traces d'exécution d'une tâche et le journal d'exécution des DAGs. Ces fichiers de journalisation sont accessible aux développeurs à travers le serveur web d'Airflow.

Conclusion

Nous avons détaillé la méthode de nettoyage des données du client dans ce chapitre, ainsi que les problèmes que nous avons l'intention de résoudre dans la solution précédente. Nous avons justifié notre sélection d'Airflow comme notre gestion de flux de travail et fourni des exemples de notre implémentation.

Chapitre 4

Tableau de bord analytique

Introduction

Dans ce chapitre, nous allons d'abord décrire l'architecture logique du micro-service de tableau de bord. Nous passerons en revue les classes qui collaborent pour fournir un service unifié, comment nous surmontons les contraintes de performances et comment nous construisons l'interface utilisateur.

4.1 Architecture Logique du microservice de tableau de bord

Comme c'est le cas pour chaque logiciel que nous avons développé jusqu'à présent, nous commençons par réfléchir à la façon dont nous pourrions diviser une logique complexe en éléments autonomes plus petits et plus faciles à implémenter qui suivent les principes de conception SOLID[9].

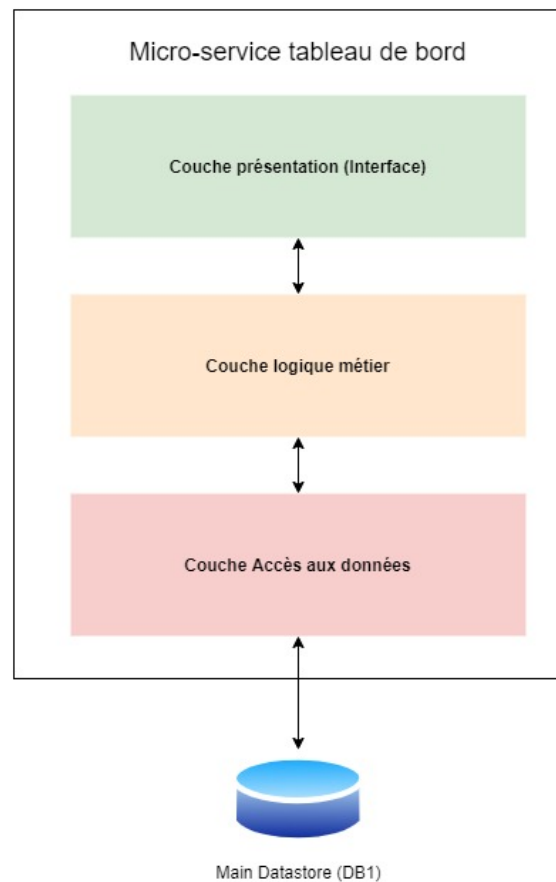


FIG. 4.1 : Architecture Logique du MS du tableau de bord

La conception à trois couches, montré dans la figure 4.1 est une architecture relativement répandue qui a résisté à l'épreuve du temps :

- **La couche de présentation** : composée d'objets qui gèrent les interactions avec le système et répondent aux événements externes. Dans un contexte d'application principale, cette couche contient des objets qui gèrent l'interception des appels d'API, l'analyse des URL et le transfert des requêtes HTTP aux gestionnaires associés, ainsi de suite.
- **La couche logique métier** : Contient des classes et des objets qui modélisent le domaine de l'utilisateur. Il gère les règles métier, les calculs et la logique qui régissent le fonctionnement d'une application.
- **La couche accès aux données** : La couche d'accès aux données connecte la couche métier avec les sources de données. Elle abstrait l'accès au moteur de base de données

r  el et fournit une interface pour interroger les donn  es de mani  re pertinente pour la couche m  tier.

4.2 Diagramme de classes

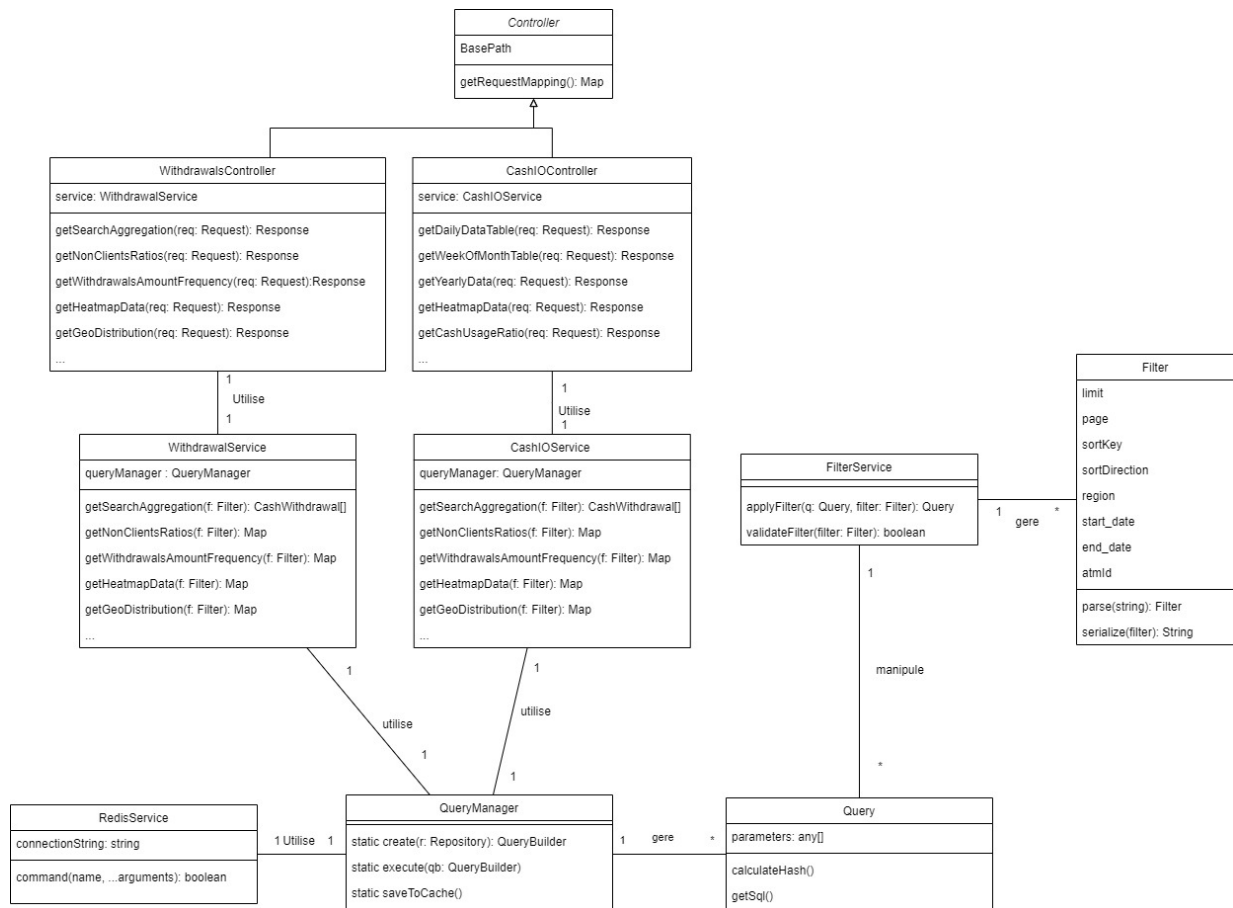


FIG. 4.2 : Diagramme de classes du micro-service tableau de bord

La liste des classes importantes impliqu  es dans le fonctionnement interne du micro-service de tableau de bord :

Note : Le diagramme ne montre pas les classes d'entit  s qui repr  sentent les donn  es de la base de donn  es, car il s'agit d'un simple mappage des tuples d'une table vers un POJO (Plain Old Java Object).

Couche	Classe	Description
Couche présentation	Controller	C'est une classe générale qui contient une logique commune à tous les contrôleurs. Les contrôleurs sont des classes qui écoutent les requêtes HTTP entrantes et les transmettent à des middlewares ou à des fonctions qui finissent par renvoyer une réponse HTTP au client.
	CashIOController et WithdrawalsController	Les deux classes étendent la classe de contrôleur et définissent des fonctions qui répondent aux requêtes client. Les contrôleurs ne contiennent pas de logique métier, ils sont uniquement responsables de l'extraction des paramètres de la requête entrante (à partir des headers, body, ou chemin de la requête) et appeler les méthodes adéquates sur les classes de service.
Couche logique métier	Filter	Filter est une classe simple qui contient des attributs utilisés pour appliquer des filtres aux requêtes de données. Les objets de cette classe sont analysés à partir des corps de requête et transformés en instructions SQL cohérentes par le service de filtrage
	FilterService	C'est un service simple qui est utilisé par d'autres services pour valider la sémantique d'un filtre ou appliquer un filtre à une requête.
	CashIOService et WithdrawalService	Ces classes contiennent des implémentations concrètes d'exigences et de règles métier. Les fonctions de ces classes sont appelées par les contrôleurs pour gérer les requêtes ou par d'autres services pour effectuer des traitements intermédiaires.

Couche d'accès aux données	Query	C'est une abstraction d'une requête SQL. Il fournit une API simple qui suit le modèle du décorateur pour créer des requêtes extensibles. Un objet de requête peut être transformé en instructions SQL exécutables ou haché pour créer une clé qui serait utilisée par le QueryManager pour mettre en cache le résultat de requêtes identiques.
	QueryManager	Utilisé par les principaux services pour créer et exécuter des requêtes. Il stocke automatiquement les résultats des requêtes dans une couche de cache pour alléger la charge de la base de données et optimiser les performances.
	Redis Service	Gère le cache. Il est utilisé par le gestionnaire de requêtes pour exécuter des commandes sur la base de données de cache.

TAB. 4.1 : Tableau explicatif des classes principales du micro-service tableau de bord

4.3 Caching et performance

Nous prévoyons que notre base de données va croître rapidement, avec vingt mille nouveaux tuples de données chaque jour, la vitesse des requêtes et la consommation de ressources en prendront certainement un coup. Pour cette raison, nous avons ajouté des composants supplémentaires qui gèrent la mise en cache des données.

4.3.1 Base de données volatile : Redis

Redis est une base de données en mémoire vive. Les valeurs dans Redis peuvent être des structures de données telles que des chaînes de caractères, hashmaps, listes, sets, sorted sets, etc. Toutes les données sont stockées dans la RAM, ce qui rend l'accès très rapide. Redis fournit également des options pour la persistance automatique des données sur le disque. Nous avons utilisé Redis pour optimiser la vitesse des requêtes et minimiser la charge d'utilisation de la base de données.

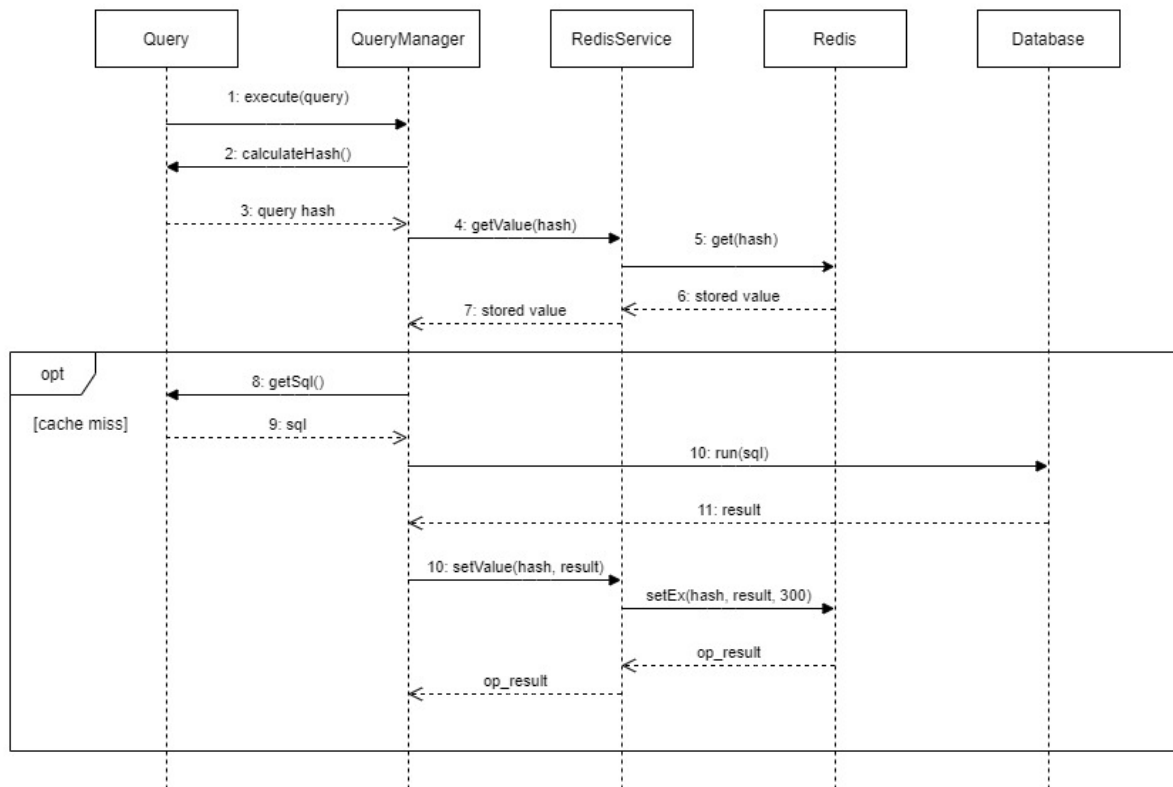


FIG. 4.3 : Diagramme de séquence d'une opération d'exécution de requête

La figure 4.3 montre le processus d'exécution d'une requête dans le service de tableau de bord. Lorsqu'un appel d'exécution d'une requête est envoyé au QueryManager, il commence d'abord par calculer le hash de la requête, ce hash est utilisé comme clé pour récupérer le résultat de la requête à partir de Redis. Si on n'a aucune clé qui correspond au hash de la requête (Cela signifie que la requête n'a pas été exécutée depuis au moins cinq minutes), on exécute le code SQL de la requête dans la base centrale et on sauvegarde le résultat dans le cache. Un appel à la commande Redis SETEX (SET et EXpire) est émis pour mettre en cache les données pendant cinq minutes, après lesquels les données sont automatiquement effacées du cache pour éviter de congestionner la mémoire.

4.4 Réalisation

Au cours de cette section, nous allons présenter la réalisation de ce sprint. Les figures suivantes illustrent les tableaux de bord contenant les données de retrait et remboursements selon plusieurs modèles (quotidien, hebdomadaires, mensuels et annuels.). Pour mettre les

données pertinentes en valeur on a essayé d'utiliser plusieurs types de chartes graphiques et des heatmaps avec différents niveaux de couleurs.

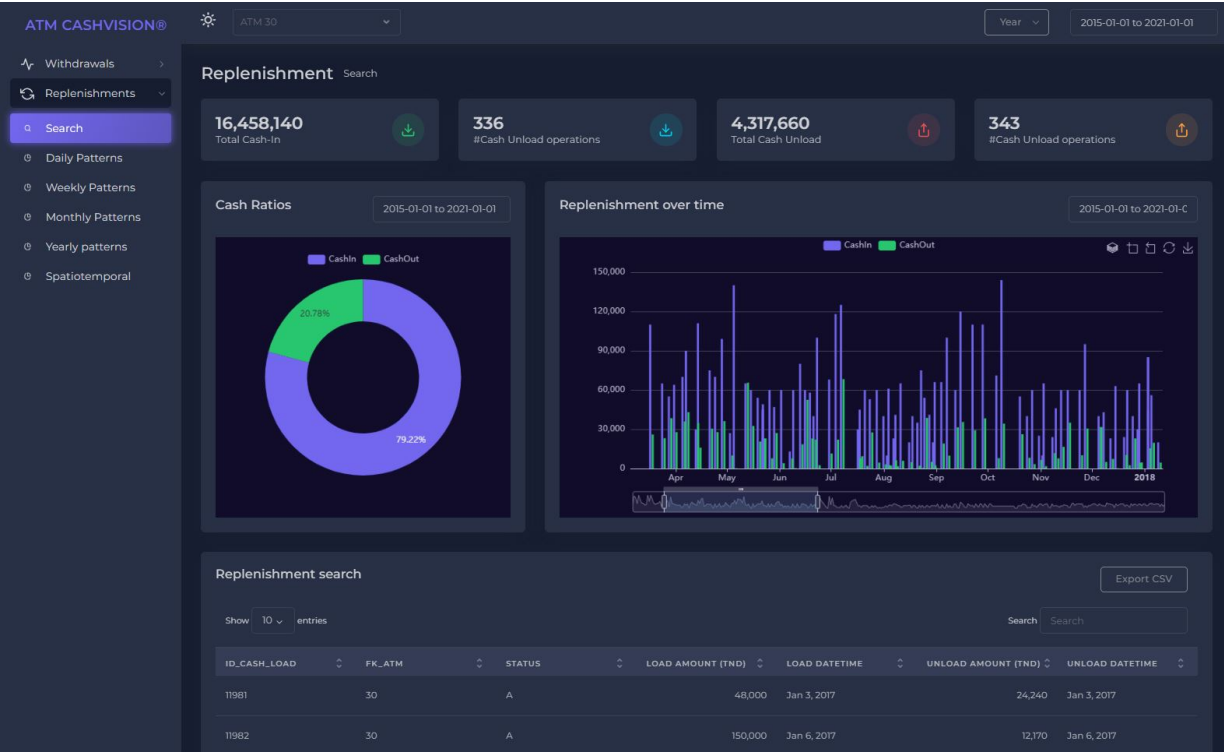


FIG. 4.4 : Tableau de bord spécifique aux opérations de remboursements



FIG. 4.5 : Le tableau de bord spécifique aux opérations de retrait hebdomadaires - 1



FIG. 4.6 : Le tableau de bord spécifique aux opérations de retrait hebdomadaires - 2

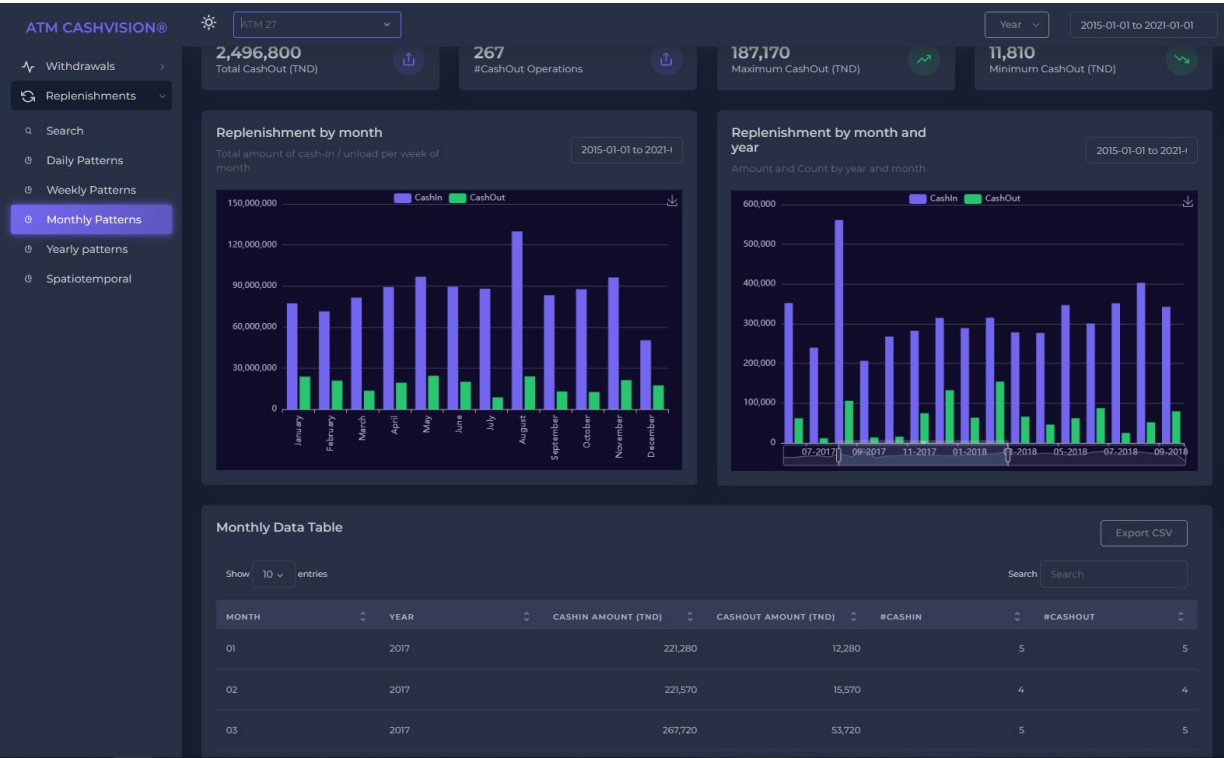


FIG. 4.7 : le tableau de bord relatif aux opérations de remboursement mensuelles

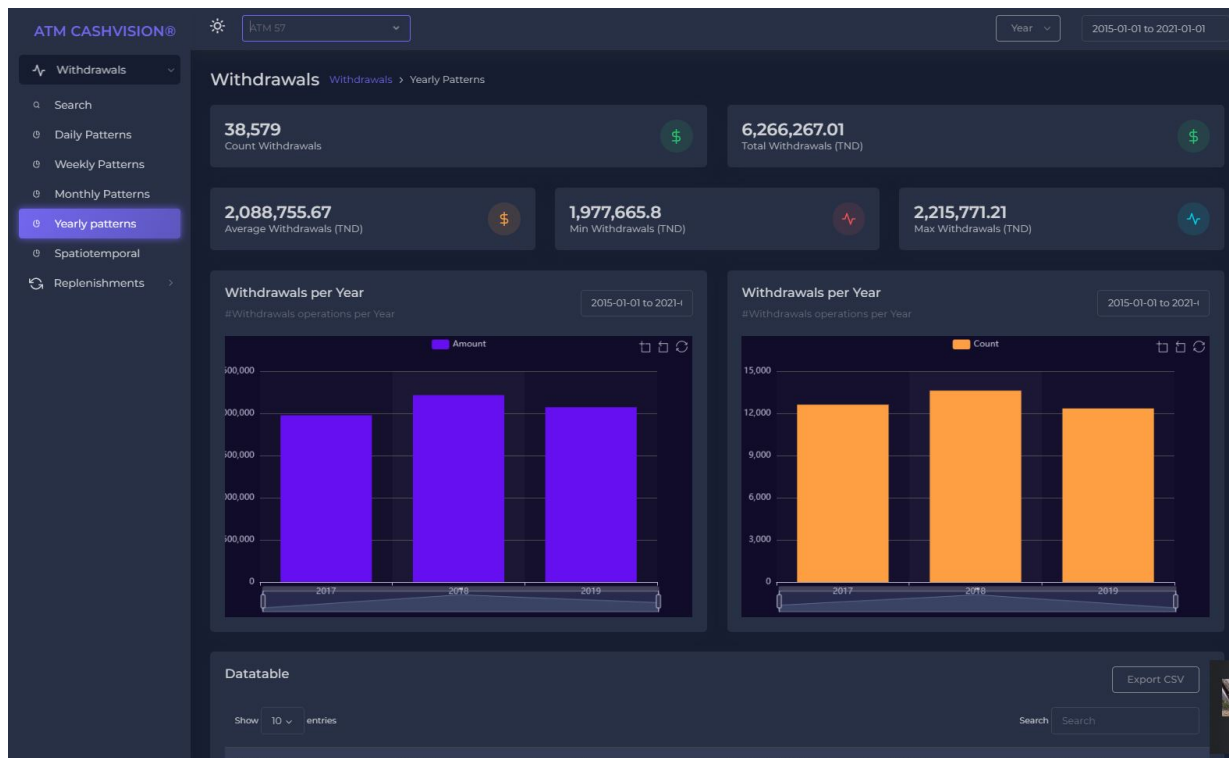


FIG. 4.8 : le tableau de bord spécifique aux opérations de retrait annuel

Conclusion

Dans ce chapitre, nous avons discuté de la conception du micro-service de tableau de bord et de la manière dont nous avons abordé les contraintes de performances liées à la récupération des données

Chapitre 5

Création des chaînes CI/CD

Introduction

Nous commencerons ce chapitre en présentant les pratiques DevOps et les valeurs qu'elles apportent à une équipe de développement logiciel. Ensuite, nous parlerons de nos sélections d'outils, puis nous examinerons deux exemples de pipelines CI/CD distincts que nous avons intégré dans notre projet.

5.1 Les pratiques DevOps

DevOps (un portemanteau de « développement » et « opérations ») est un ensemble de pratiques et outils utilisé par les équipes de développement logiciel pour augmenter leurs capacités à livrer des logiciels de qualité d'une manière plus rapide que la méthode traditionnelle, dans laquelle les équipes développement et opérations sont séparés. Dans les pratiques DevOps l'équipe est responsable a la fois du développement ainsi que le déploiement des logiciels.

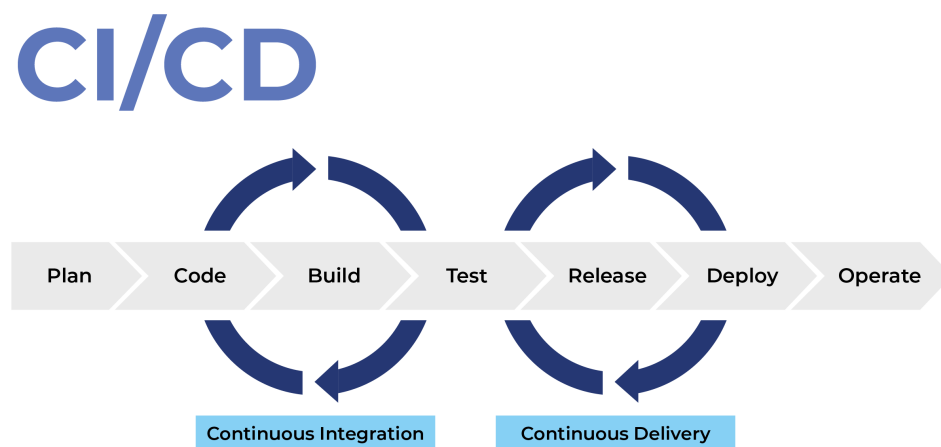


FIG. 5.1 : La pratique DevOps

La figure 5.1 montre le processus de développement itératif au cœur de la pratique

DevOps. On peut catégoriser ces pratiques en deux familles :

- **L'intégration continue (CI)** : Consiste à automatiser l'intégration des modifications de code de plusieurs contributeurs dans un seul projet logiciel. Ça permet aux développeurs de fusionner fréquemment les modifications de code dans un référentiel central où la compilation et les tests s'exécutent ensuite automatiquement.
- **Le déploiement continu (CD)** : Il s'agit d'automatiser le processus de déploiement de logiciels pour permettre la livraison d'un logiciel par petits incréments. Cela permet aux équipes d'être plus productives et de recevoir plus rapidement le feedback des clients.

5.2 Outils

Comme la pratique du génie logiciel a évolué, les outils existants se sont améliorés et de nouveaux sont apparus. Dans cette section on discutera les technologies qu'on a choisies pour la mise en œuvre de nos pipelines CI/CD.

5.2.1 Git et GitHub

Typiquement, le développement de logiciels est un processus collaboratif dans lequel plusieurs développeurs travaillent sur la même base de code. Nous avons besoin d'outils pour synchroniser le travail entre les développeurs afin d'éviter les conflits d'édition. Les logiciels de contrôle de source résolvent ce type de problèmes, le plus pertinent de ces logiciels dans l'industrie est s'appelle "Git", développé par le créateur de Linux.

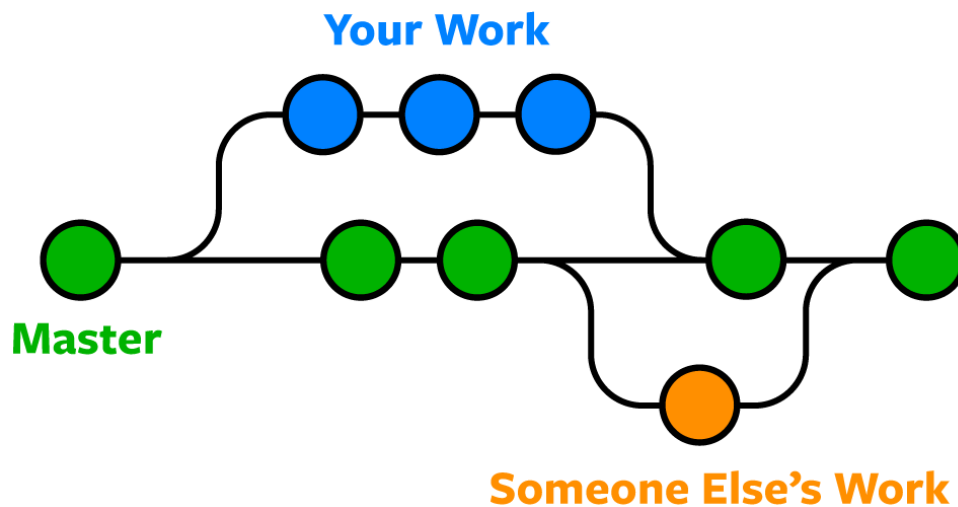


FIG. 5.2 : Représentation graphique de l'évolution d'un référentiel git

Ce qui distingue git des autres logiciels de contrôle de source, sont ses fonctionnalités de branchement et fusion de code[10]. une branche est une copie du code qui évolue séparément du code principal et qui peut être fusionné a tout moment avec ce dernier. Cette option encourage l'expérimentation sans soucis et permet de garder toujours une version robuste du code. Dans la figure 5.2 les nœuds représentent les états du code. Un commit permet de changer un code d'un état a un nouvel état. Les nœuds en bleu et en orangé représentent des branches parallèles à la branche principale Master. La fusion d'une branche A avec une autre branche B créer un commit qui intègre les changements de A dans B.



FIG. 5.3 : Logo de GitHub

GitHub est une filiale de Microsoft qui propose un service d'hébergement de référentiel Git basé sur le cloud. Essentiellement, cela permet aux individus et aux équipes d'utiliser Git beaucoup plus facilement pour le contrôle de version et la collaboration. En plus d'un service d'hébergement de référentiel git, GitHub offre beaucoup plus d'autres fonctionnalités attirantes telle que les outils de suivi des problèmes intégrable avec le code source d'un projet, un espace de stockage de conteneurs Docker et d'outils de création de pipelines CI/CD (GitHub Actions).

5.2.2 Serveur de CI/CD

Un serveur CI/CD est un logiciel qui s'intègre avec les outils de contrôle de code source pour permettre l'exécution des processus de tests et compilation du projet automatiquement. Nous avons déjà décidé d'utiliser GitHub pour l'hébergement de nos projets, l'intégration de leur service de CI/CD, appelée GitHub Actions, sera facile et même plus avantageuse que déployer notre propre solution.

5.2.3 DockerHub

Docker Hub[11] est un service fourni par Docker pour rechercher et partager des images de conteneurs. Il s'agit du plus grand référentiel d'images de conteneurs contenant des images publiques de presque tous les logiciels open-source ou autre. Ce service offre aussi un espace de stockage d'images privé qu'on va utiliser à la phase de déploiement pour assurer la livraison de nos programmes d'une façon sécurisée.

5.3 Pipeline CI/CD d'un microservice

Nos dépôts Git possèdent deux branches « dev » et « master ». Le développement se déroule dans la branche dev et à chaque fois qu'on a besoin de publier une version, on fusionne la branche dev avec master. La fusion des deux branches déclenche un enchaînement qui exécute des tests, conteneurise et déploie le micro-service dans le serveur de production

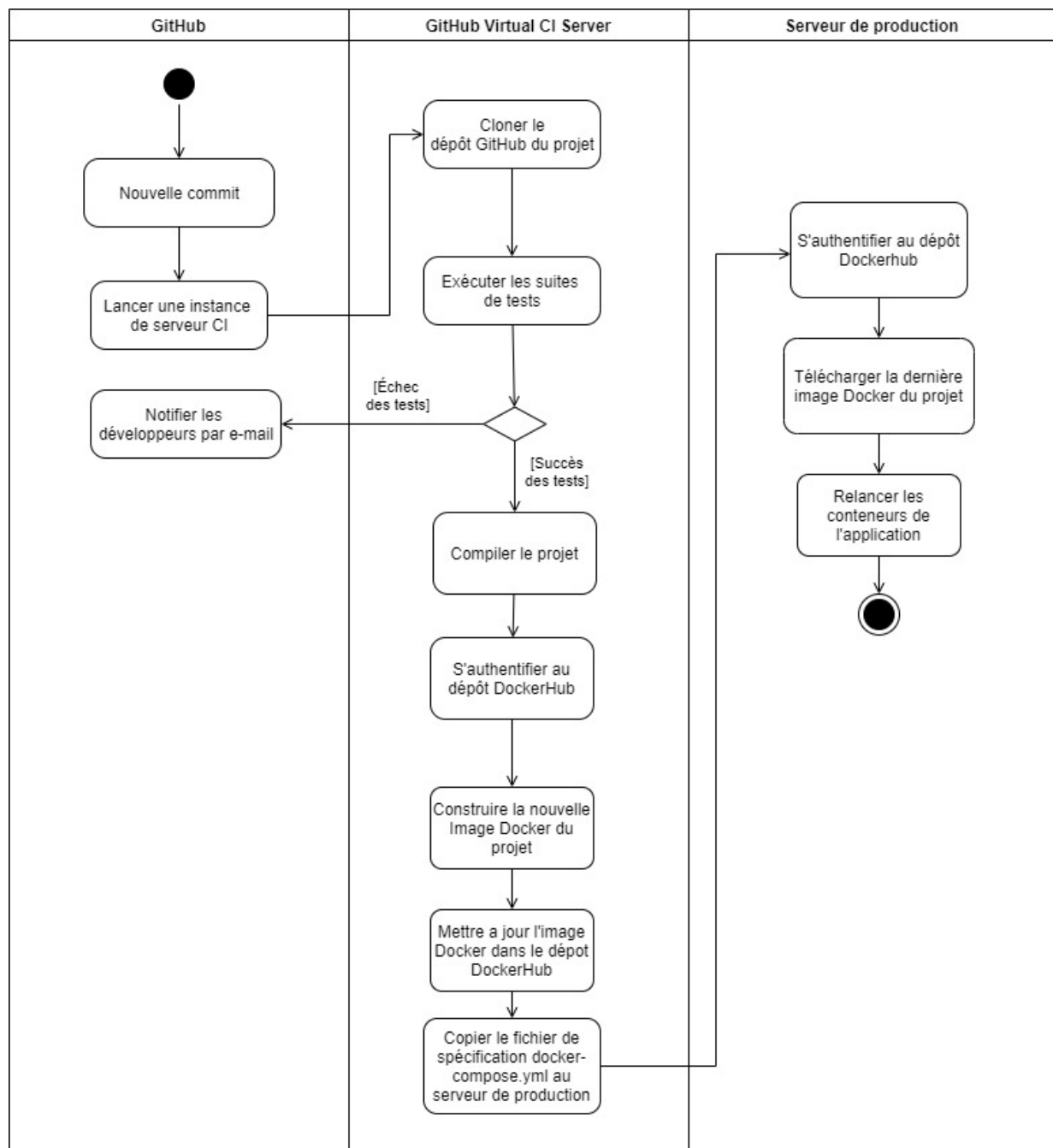


FIG. 5.4 : Diagramme d'activité d'une chaîne de CI/CD de micro-service

Le diagramme d'activité de la figure 5.4 montre un exemple de pipeline CI/CD pour un micro-service. Le pipeline s'exécute sur une machine virtuelle créée par GitHub. Après le clonage du dépôt git, la machine virtuelle exécute l'ensemble de tests unitaires, d'intégration et "end-to-end". Si tous ces derniers s'exécutent avec succès, on passe à la compilation

et conteneurisation du projet. Le résultat de cette étape est une image prête à déployer sur le serveur. Au lieu de copier l'image docker directement au serveur, on la sauvegarde d'abord dans le référentiel DockerHub, puis on connecte au serveur via SSH pour la télécharger et instancier. De cette façon on garde un historique de versions d'images et on peut retourner vers une version antérieure en cas d'erreur.

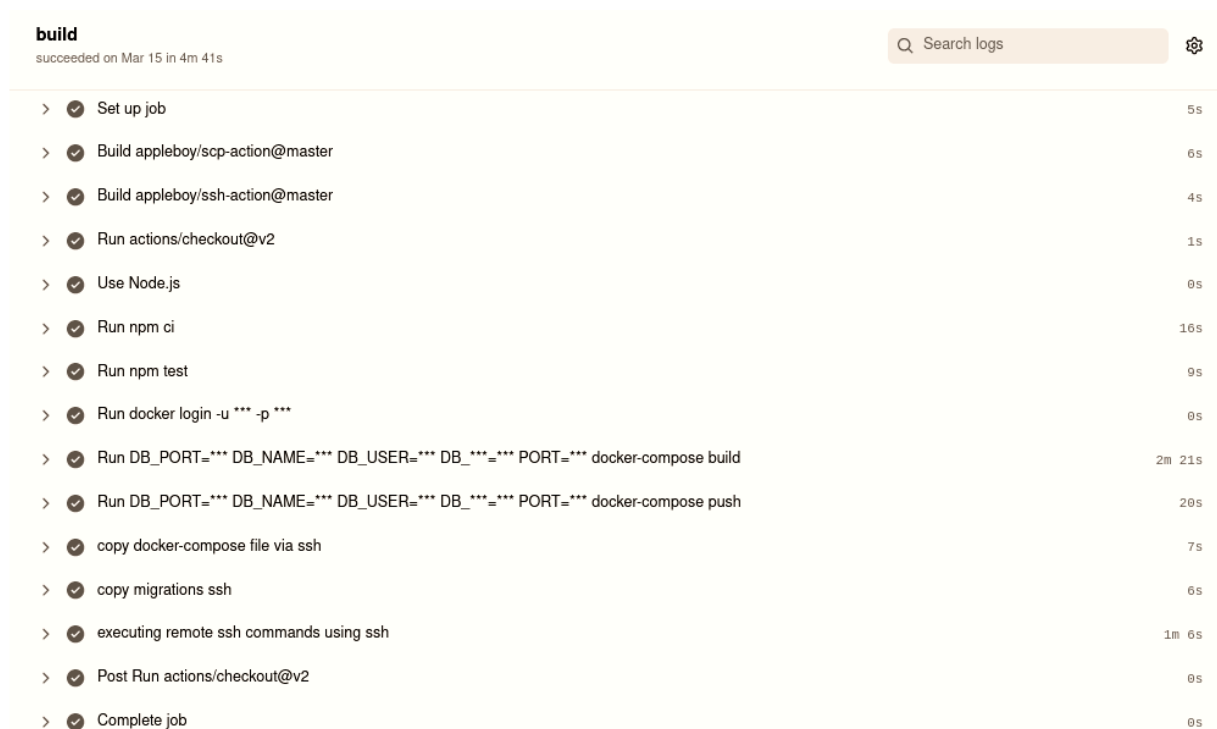


FIG. 5.5 : Capture d'écran du log d'exécution d'un pipeline CI/CD pour un micro-service

On peut observer l'avancement de la chaîne étape par étape grâce à GitHub. Il est à noter qu'à chaque étape du pipeline, tous les messages issus des opérations exécutées sont repris dans une interface spécifique dans le site GitHub comme le montre la figure 5.5. Il est aussi à bien noter que si une seule opération échoue, toute la chaîne s'arrête et le développeur sera notifié avec e-mail.

5.4 Migrations automatiques des bases de données

Les schémas de base de données définissent la structure et les relations entre les données. Malgré qu'il est important de développer un schéma bien pensé au début de nos projets, l'évolution des exigences rend les modifications de notre schéma impossibles à éviter.

5.4.1 Problématique

Le schéma d'une base de données gère la forme et les contraintes des informations enregistrées. On rencontre un problème lorsque plusieurs personnes travaillent sur une base de données partagée dans une architecture micro-service. Chaque développeur disposera d'une copie de la base de données sur sa machine locale qui évoluerait séparément des copies d'autres développeurs et étant donné la délicatesse des migrations, cela peut poser des conflits qui pourraient nous faire perdre des données.

5.4.2 Solution proposée

Le problème d'évolution du schéma de la base de donnée est très similaire en nature au problème d'évolution de code source. Ce problème est résolu à l'aide des outils de contrôle de code source qu'on a discuté précédemment. Puisqu'un schéma de base de données peut être représenté sous forme de code dans la syntaxe data description language (DDL) de SQL. Notre solution consiste à combiner Git et Flywaydb, un outil de gestion de migrations, pour résoudre au problème d'évolution de la base de données

5.4.3 Outil de migration FlywayDb

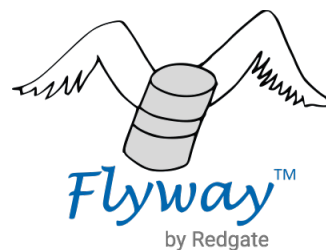


FIG. 5.6 : logo de FlywayDb

FlywayDb[12] est un simple outils de gestion des migrations des bases de données, écrit en Java et peut être utilisé séparément des frameworks de développement logiciel. Il permet le versionnage de la base de données via des fichiers SQL bien structurés, fournit des fonctionnalités de restauration automatique en cas d'erreurs et possède des mécanismes de tests les fichiers de migration.

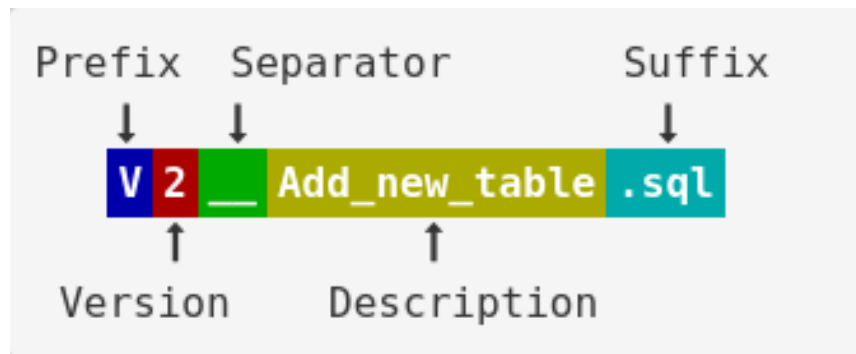


FIG. 5.7 : Nomenclature des fichiers de migration

Une migration dans Flyway est un simple fichier qui contient du code SQL, généralement des instructions de la syntaxe DDL. Ces fichiers doivent être nommés dans une façon bien spécifique pour être considéré par le programme. La figure 5.7 montre un exemple de fichier de migration ; les noms des fichiers doivent commencer avec une lettre "V" puis un numéro qui indique l'ordre d'exécution du fichier.

flyway_schema_history Enter a SQL expression to filter results (use Ctrl+Space)										
	installed_rank	version	description	type	script	checksum	installed_by	installed_on	execution_time	success
1	1	1	create db1	SQL	V1_create_db1.sql	828,598,572	atmcashv_user	2021-03-17 17:15:01	6,666	[v]
2	2	2	update data db1	SQL	V2_update_data_db1.sql	927,787,096	atmcashv_user	2021-03-17 17:15:08	780	[v]
3	3	3	rename cols	SQL	V3_rename_cols.sql	1,030,065,010	atmcashv_user	2021-03-24 15:33:22	517	[v]
4	4	4	create ai tables	SQL	V4_create_ai_tables.sql	85,502,290	atmcashv_user	2021-03-24 15:35:01	1,099	[v]
5	5	5	add ai pipeline columns	SQL	V5_add_ai_pipeline_columns.sql	1,308,649,176	atmcashv_user	2021-04-01 15:30:02	423	[v]
6	6	6	ai pipeline changes	SQL	V6_ai_pipeline_changes.sql	1,926,283,071	atmcashv_user	2021-05-17 15:40:58	35,265	[v]
7	7	7	changes order columns	SQL	V7_changes_order_columns.sql	203,605,462	atmcashv_user	2021-05-17 15:58:58	591	[v]
8	8	8	prediction tables	SQL	V8_prediction_tables.sql	-1,742,926,469	atmcashv_user	2021-05-21 20:57:49	933	[v]

FIG. 5.8 : Historique des migrations

Pour garder une trace de la version de schéma de la base de données, Flyway crée une table supplémentaire qui contient toutes les migrations qui ont été exécutées. La figure 5.8 montre un exemple de la structure et les données de cette table. Chaque ligne contient le nom de fichier et un hash de la somme de contrôle qui permet de détecter si un fichier de migration a été modifié.

5.5 Pipeline CI/CD de migration automatique de base de données

Après la création d'un référentiel git qui hébergera nos fichiers de migration, il nous reste que définir le pipeline qui, à l'aide de Flyway, testera et exécutera automatiquement les fichiers SQL dans la base de production.

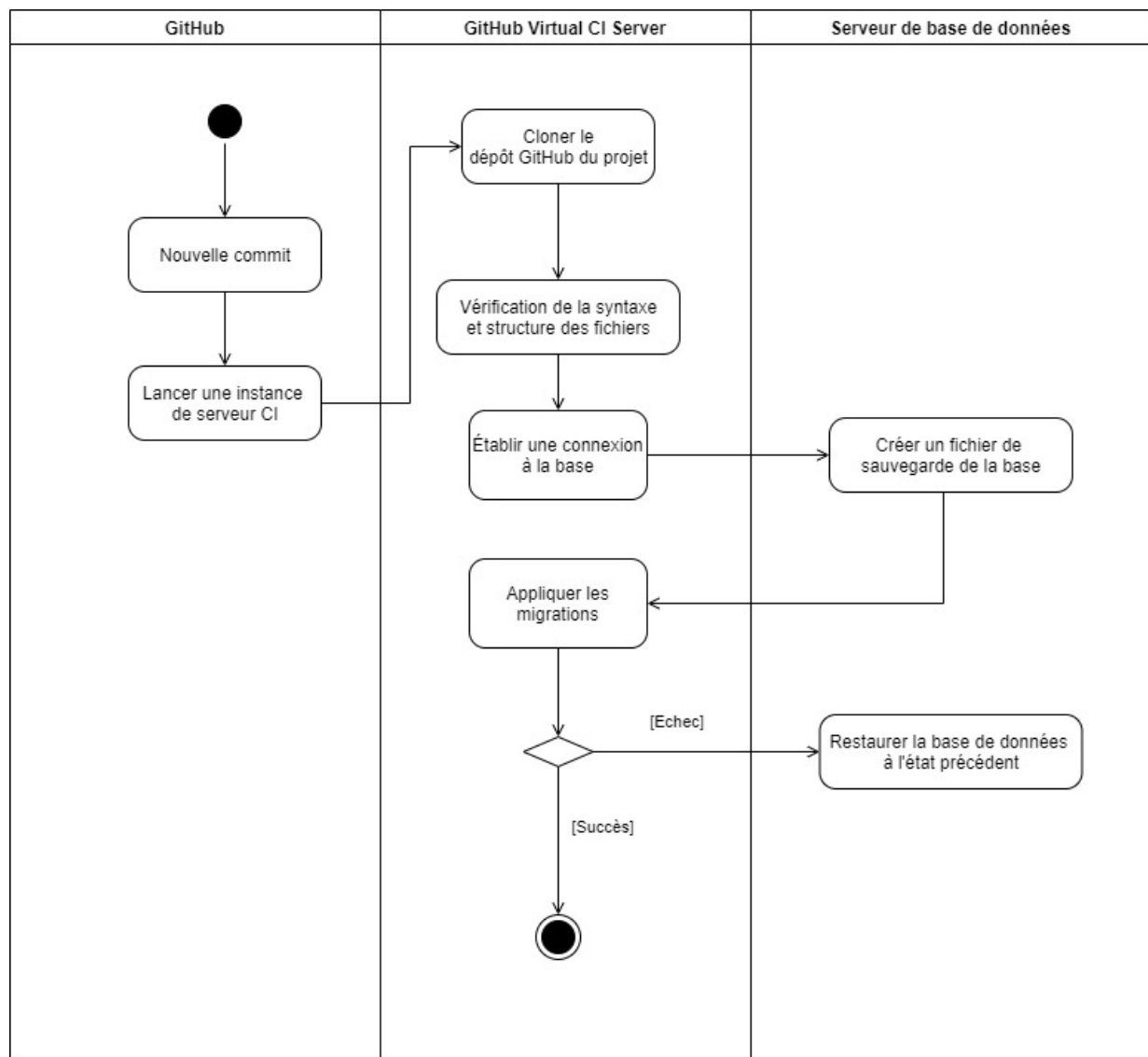


FIG. 5.9 : Diagramme d'activité du pipeline CI/CD de la migration de base de données

Notre pipeline de migration, illustré à la figure 5.9, démarre lorsqu'un nouveau commit est poussé vers le référentiel qui héberge le code source de notre schéma. GitHub commence par instancier un serveur CI/CD et y cloner le code. Nous avons docker préinstallé sur ce serveur, nous en profitons donc pour mettre en place FlywayDB. La première chose que ce

programme fait est de vérifier la syntaxe et la sémantique des fichiers SQL de migration, si tout est correctement écrit, il se connecte via SSH au serveur de base de données et crée un fichier de sauvegarde qui sert de point de restauration en cas de problèmes après la migration. Les migrations sont exécutées une fois le processus de sauvegarde terminé.



FIG. 5.10 : Log du pipeline auto-migration

Comme la figure 5.10 le montre, pour l'installation de Flyway dans le serveur CI on a créé une image docker qui, étant donnée les coordonnées de la base, exécute les commandes de test et migration.

Conclusion

Nous avons discuté des avantages des méthodes DevOps dans ce chapitre, puis avons examiné les options d'outils pour mettre en œuvre plusieurs pipelines CI/CD et les intégrer dans notre projet.

Chapitre 6

Surveillance de l'infrastructure

Introduction

Les spécifications du service de surveillance de l'infrastructure sont traitées dans ce chapitre. Nous comparerons les outils et solutions disponibles, puis expliquerons notre sélection technique et notre architecture, et enfin notre mise en œuvre.

6.1 L'importance de la surveillance

Malgré les tests rigoureux certains dysfonctionnements peuvent survenir après le déploiement en production. L'allocation et la libération des ressources, l'utilisation de la mémoire et du processeur et les débits de transfert de données, pour n'en nommer que quelques-unes, sont des mesures clés que nous devons surveiller afin d'avoir une idée du comportement du système et d'éviter les goulots de performance ou les problèmes potentiels.

6.2 Besoins fonctionnelles

Les fonctionnalités suivantes doivent être disponible dans le système de surveillance :

- Le système doit permettre la visualisation des métriques d'utilisation de ressources dans le serveur de production.
- Il doit permettre la surveillance de l'état de la base de données.
- Envoyer automatiquement des alertes par e-mail lorsqu'une certaine limite est atteinte.

6.3 Besoins non-fonctionnelles

Vu que notre plateforme évolue rapidement, notre futur système de surveillance doit être :

- **Facilement Extensible** : L'ajout d'autre tableau de bords doit être possible sans le besoin d'un développeur.
- **Facilement Personnalisable** : On doit être capable de choisir les métriques à surveiller
- **Sécurisé** : Le système doit être protégé contre les accès non autorisés

6.4 Solutions existantes

On a considéré les solutions les plus utilisés et recommandés à nous pour faire notre choix final :

Nom	Open Source	Intégration	Extensibilité
Nagios[13]	Oui, écrit en C et python (971 étoiles sur Github)	Installation et execution d'une application serveur qui surveille interroge périodiquement le systeme. Ce serveur possede un tableau de bords intégré.	Extensible a travers des plugins développé par la communauté pour la surveillance de différents types de systemes.
DataDog[14]	Non, Solution payante hébergée dans le cloud	Installation d'agents de surveillance qui collectent des informations sur le systeme a surveiller puis ils les envoient au serveur de DataDog pour la visualisation.	Peut etre étendu pour surveiller des systeme externes (ex : base de données) et même du code interne (surveillance des erreurs, statistiques sur l'activité du code telque le nombre des exceptions, les fonctions appelée, le temps de latence etc...).
Sentry[15]	Non, Solution payante hébergée dans le cloud	Intégration directe avec le code de l'application. Injection de code de monitoring qui communique avec les serveur de Sentry.	Le code de surveillance propriétaire de Sentry est disponible pour la majorité des langages de programmation.
Prometheus[16]	Oui, écrite en GoLang (36.9K étoiles sur Github)	Installation et execution d'une application serveur qui surveille interroge périodiquement le systeme. Les résultats d'interrogation sont enregistré dans une base de données chronologique.	Extensible a travers des agents de surveillance appelé "exporters" qui sont responsable de l'interrogation des différents systemes. Le serveur est responsable de la collecte des données a partir des exporters.

TAB. 6.1 : Tableau comparatif des solutions de surveillance

Le tableau 6.1 résume une étude comparative des technologies de surveillance d'infrastructure informatique.

Notre choix s'est limité aux solutions libres et open source, à savoir Prometheus et

Nagios. Tester chacun d'eux par rapport à nos exigences fonctionnelles nous a apporté de bons arguments pour notre technologie de choix. Nous avons fini par choisir Prometheus car c'était la solution la plus populaire parmi le groupe admettant un excellent support communautaire (bonne documentation, images Docker officielles, etc.)

6.5 Architecture Prometheus

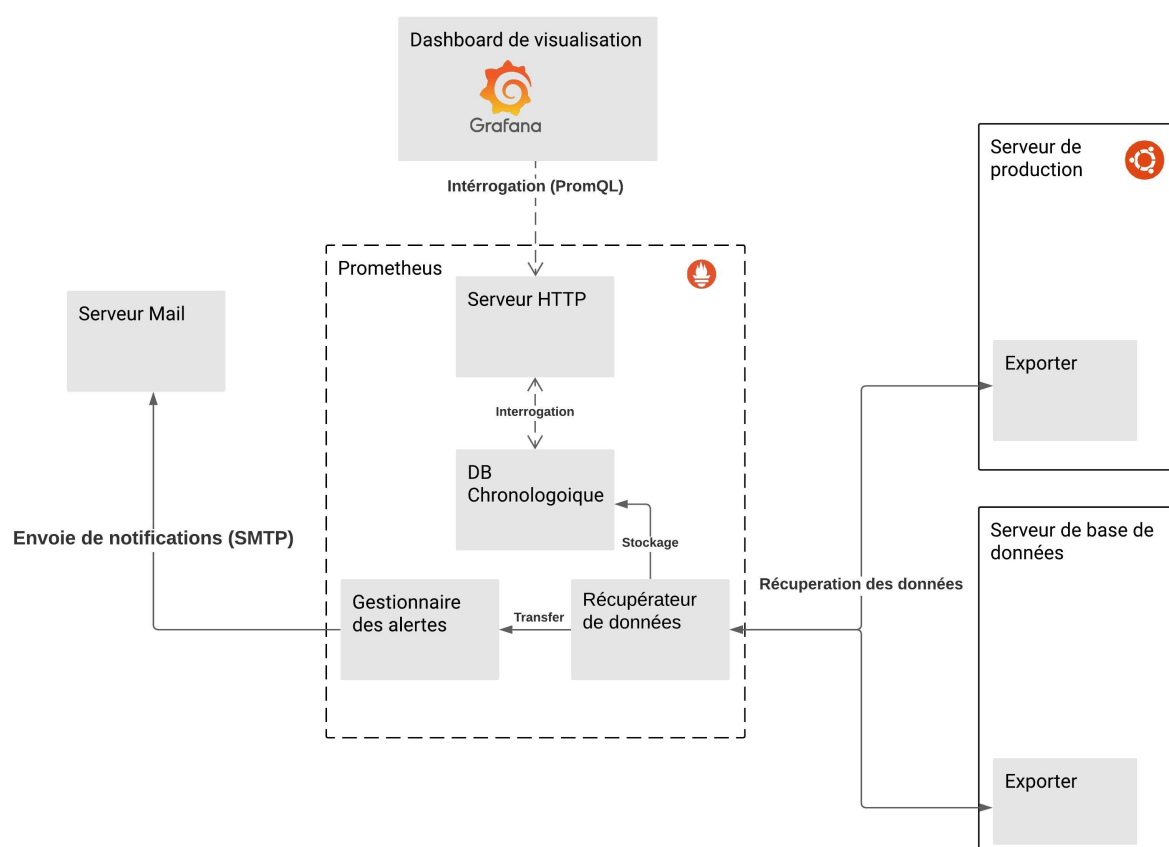


FIG. 6.1 : Architecture détaillée du système de surveillance

De nombreux sous-systèmes sont impliqués dans le processus de surveillance. La figure 6.1 est un diagramme simplifié des composants et leurs communications ensemble pour constituer un système cohérent :

- **Les Exporters** : Sont des petites applications qui s'exécutent sur les machines à surveiller et vont exposer les métriques de suivi. Il existe des exportateurs pour

les systèmes les plus couramment utilisés en génie logiciel, On a utilisé **CAdvisor** exporteur qui surveille les environnements Docker, **pg-exporter** pour le monitoring de nos bases de données Postgres et **node-exporter** pour le monitoring des serveurs de production.

- **Le récupérateur de données** : Fait partie de Prometheus, c'est un programme configurable, responsable de la récupération des données de surveillance à partir des exporteurs.
- **Base de données chronologique** : C'est la base de données interne de Prometheus, elle stocke les informations reçues à partir du récupérateur et interagit avec le serveur HTTP pour l'interrogation des données.
- **PromQL** : PromQL est un langage d'interrogation spécifique à Prometheus permettant de sélectionner et d'agréger les métriques stockées en base de données, il est particulièrement adapté aux fonctionnements avec une base de données chronologiques en fournissant de nombreuses fonctionnalités spécifiques à la manipulation du temps (décalage de temps, calcul de moyenne, maximum, etc.).
- **Gestionnaire des alertes** : Le programme responsable de déclencher l'émission d'alertes par plusieurs intermédiaires(email pour notre cas) en fonction de seuils prédéfinis par l'utilisateur. La configuration d'alertes se base sur la définition d'une expression au langage PromQL ainsi qu'une durée de temps permettant de caractériser le temps nécessaire pour déclencher une alerte.
- **Graphana** : Graphana [17] est une plateforme open source de génération de graphiques et tableaux de bord à partir de plusieurs sources de données. On a utilisé cette plateforme pour visualiser les données de surveillance.

6.6 Réalisation

Grace à Docker, on peut facilement déployer tous les composants de notre système de surveillance. On a créé nos propres images qui contiennent les composants et leurs configurations pour faciliter le déploiement sur machine. Il nous reste alors que le développement des requêtes PromQL et leur intégration avec Graphana pour la visualisation.

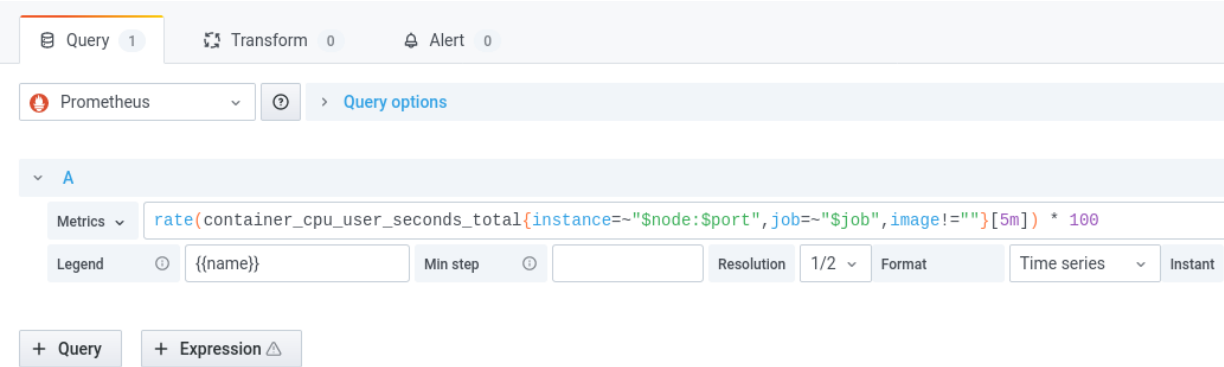


FIG. 6.2 : Exemple de requête PromQL

La figure 6.2 démontre un exemple de requête PromQL simple qui permet de calculer le temps d’usage du processeur pour chaque conteneur, périodiquement tous les cinq minutes.

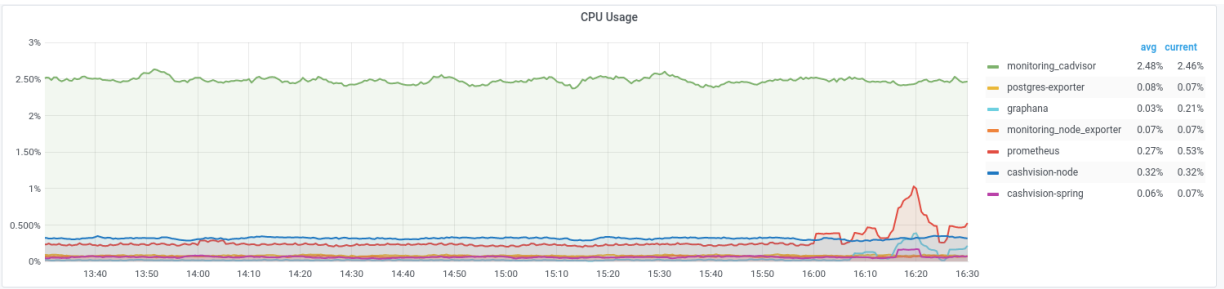


FIG. 6.3 : Visualisation de la requête de la figure 6.2

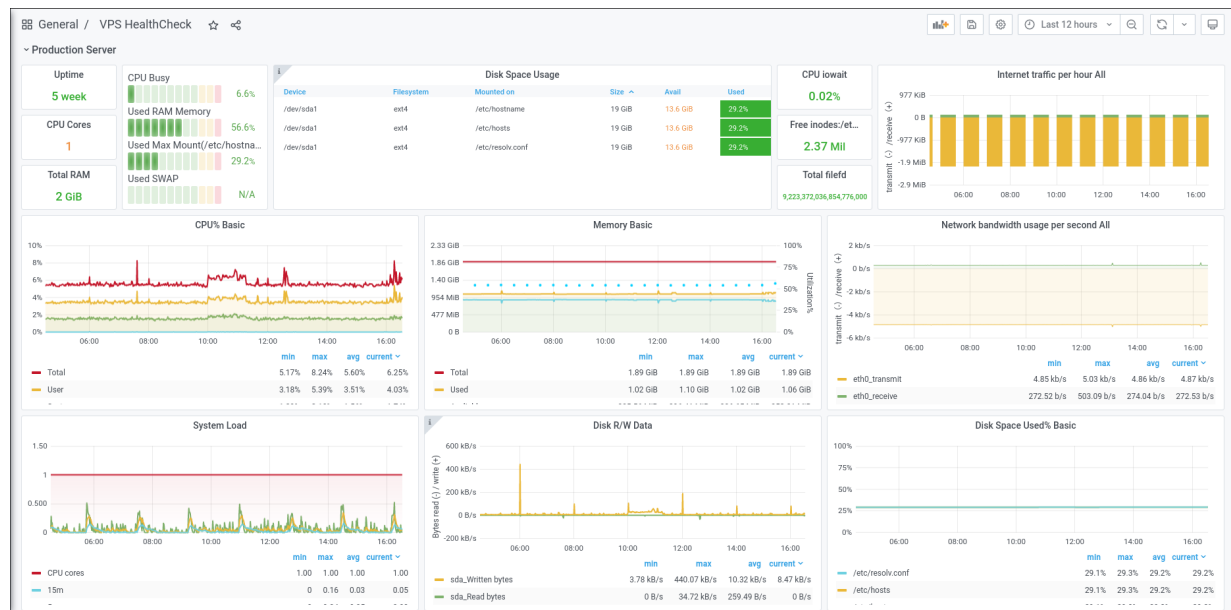


FIG. 6.4 : Tableau de bords de surveillance du serveur de production

La figure 6.4 contient le tableau de bord de surveillance d'utilisation de ressources physiques. Cette vue contient le pourcentage d'utilisation CPU mémoire et disques dans le temps ainsi que le débit d'envoi et réception d'information de l'interface réseau.

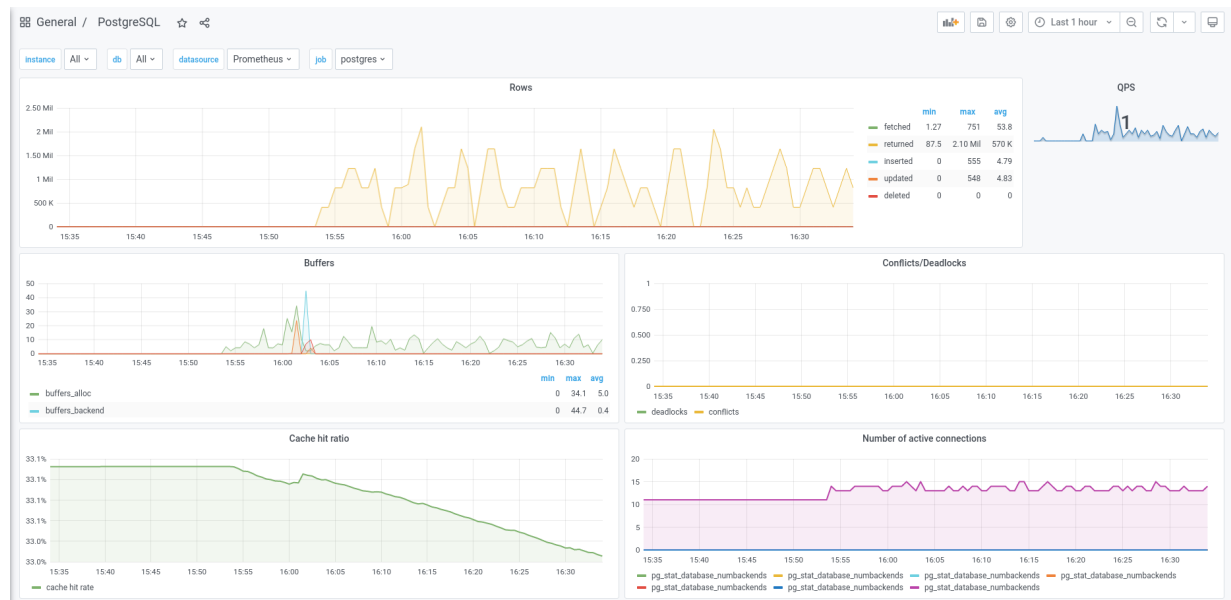


FIG. 6.5 : Tableau de bords de surveillance de la base de donnée principale

La figure 6.5 montre la visualisation des différentes métriques relatives au fonctionne-

ment de la base de données tel que le nombre de tuples récupérés insérés, mises à jour et supprimées, le nombre de connexions actives et le nombres de requêtes par seconde



FIG. 6.6 : Tableau de bords de surveillance des conteneurs Docker

Nous avons créé le tableau de bord de la figure 6.6 pour surveiller les conteneurs en exécution. On s'intéresse aux pourcentages d'utilisation du processeur et mémoire pour chaque conteneur.

Conclusion

Dans ce chapitre, nous avons discuté les besoins du service de surveillance, évalué les outils et technologies actuels et justifié notre choix. De plus, nous avons présenté l'architecture et la mise en œuvre de notre solution.

Chapitre 7

Sécurité des microservices

Introduction

Ce chapitre se concentrera sur la protection de nos micro-services. Nous commencerons par décrire les problèmes de sécurité qui surviennent avec une architecture de micro-service, puis nous présentons notre solution. Nous passerons en revue les techniques qu'on a utilisées ainsi que la manière dont nous mettons en place des procédures d'authentification classiques.

7.1 Problématique

7.1.1 Défis de sécurité dans une architecture micro-service

Dans une application monolithique traditionnelle, l'authentification et l'autorisation sont, développés dans un module qui peut être utilisée pour contrôler l'accès à toutes les APIs de l'application. Dans ce contexte, rendre un service protégé (ou limitée à certains utilisateurs) revient à ajouter un "garde" à cette interface. En ce qui concerne les micro-services, la mise en œuvre d'une couche de sécurité est un peu plus difficile : comment l'état d'authentification va être persisté à travers les sessions, comment un micro-service pourrait-il savoir si un utilisateur admet les privilèges d'accès au service, qui doit gérer l'authentification des utilisateurs, etc.

7.1.2 Anti-pattern d'implémentation

Certainement, ce que nous ne voulons PAS faire, c'est implémenter l'authentification et l'autorisation dans chaque micro-service dont nous disposons. Cela prendrait beaucoup de temps et demandera une charge de travail énorme lorsque nous voulons mettre à jour nos mécanismes de sécurité.

7.2 Solution proposée

Notre solution consiste à créer un seul micro-service de sécurité pour gérer les utilisateurs et contrôler l'accès. Étant donné que chaque requête adressée à nos backends passe par la passerelle API implémentée en un serveur NGINX, nous avons la liberté de contrôler le flux des requêtes comme nous le souhaitons. Comme le montre la figure 2.5, le micro-service de sécurité communique directement avec la passerelle API pour autoriser toutes les requêtes de passage et il peut également les modifier pour ajouter des informations supplémentaires sur l'utilisateur.

7.3 Conception

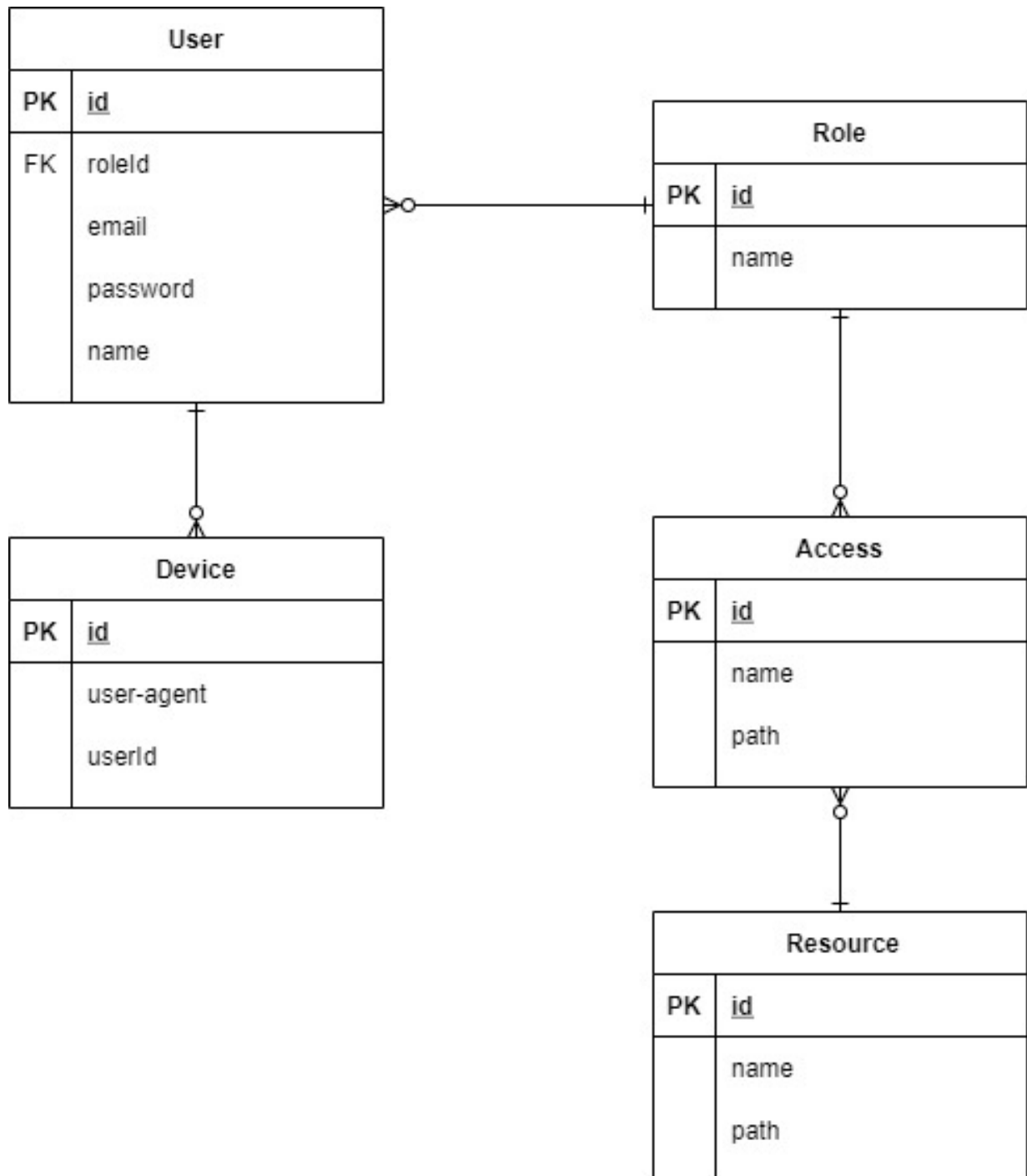


FIG. 7.1 : Diagramme d'entité-relation du service d'authentification

Nous utilisons les entités de la figure 7.1 pour modéliser nos exigences système

Entité	Attribut	Description
User	id	Un identifiant universel unique (UUID). Dans une base de données, il sert de clé primaire pour la table utilisateur
	email	Email de l'utilisateur
	name	Nom de l'utilisateur
	password	Hash du mot de passe de l'utilisateur
Resource	id	identifiant universel unique de la ressource
	name	Désignation de la ressource. Nom significatif attribué par l'administrateur du système à une ressource
	path	Path est une chaîne de caractères qui contient le chemin qui permet d'accéder à un endpoint d'un service à partir de l'API gateway. Cet attribut est utilisé pour valider l'accès à cet endpoint.
Role	id	Identifiant unique du rôle
	name	Désignation du rôle
Device	userId	Identifiant unique de l'utilisateur
	userAgent	Chaîne de caractères qui permet d'identifier quel navigateur est utilisé, quelle version et sur quel système d'exploitation
Access	userId	Identifiant unique de l'utilisateur
	resourceId	Identifiant unique de la ressource

TAB. 7.1 : Tableau descriptif des entités du système

7.4 Outils

Le processus d'authentification implique plusieurs protocoles et technologies. Dans cette section nous présenterons les outils que nous avons utilisés pour atteindre notre objectif.

7.4.1 Json Web Token (JWT)

JSON Web Token (JWT)[18] est une norme ouverte qui propose une méthode brève et autonome pour la sécurisation des informations échangés sur un réseau l'envoi. Comme elles sont signées numériquement, l'intégrité de ces données peut être facilement vérifiée. Les jetons JWT sont divisés en trois composants séparés par des points (.).

The image shows a web-based JWT decoding tool. At the top, there is a dropdown menu labeled 'ALGORITHM' with 'HS256' selected. Below this, the tool is divided into two main sections: 'Encoded' and 'Decoded'.

Encoded: This section displays the raw JWT token: `eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoiYWRtaW4iOnRydWV9.TJVA95OrM7E2cBab30RMhrHDcEfxjoYZgeFONFh7HgQ`.

Decoded: This section shows the decoded components of the token:

- HEADER:** A JSON object: `{ "alg": "HS256", "typ": "JWT" }`
- PAYLOAD:** A JSON object: `{ "sub": "1234567890", "name": "John Doe", "admin": true }`
- VERIFY SIGNATURE:** This section shows the process of verifying the signature. It displays the HMACSHA256 function being applied to the base64 encoded header and payload, using a secret key. The secret key is shown in a text input field with the value 'secret'. The final output is: `HMACSHA256( base64UrlEncode(header) + "." + base64UrlEncode(payload), secret )`.

FIG. 7.2 : Décodage d'un token JWT

La figure 7.2 montre le résultat de décodage et la structure d'un token JWT

- **Header** : C'est l'en-tête du jeton. Elle se compose généralement de deux parties : le type de jeton (typiquement JWT), et la technique de signature utilisée, qui est soit HMAC SHA256 ou RSA.

- **Payload** : C'est la charge utile du jeton, le payload contient les informations échangées ainsi que quelques informations supplémentaires telles que les dates de création et expirations des jetons.
- **Signature** : La signature est utilisée pour garantir que le message n'a pas été falsifié, et dans le cas de jetons signés avec une clé privée (RSA)[19], elle peut également garantir que l'expéditeur du JWT est bien celui qu'il prétend être.

7.4.2 Signatures numériques

Les signatures numériques utilisent des algorithmes à clé publique pour assurer l'intégrité des données. Lorsqu'on signe des données avec une signature numérique, quelqu'un d'autre peut vérifier la signature et prouver que les données proviennent de nous et n'ont pas été modifiées après que nous les avons signées.

7.4.3 Hachage cryptographique

Le hachage est un processus cryptographique qui transforme tout type de données en une chaîne de texte unique. Le hachage de n'importe quelle donnée donnera toujours une chaîne de même taille et il est quasi impossible de pas récupérer les données d'origine à partir d'un hash.

7.4.4 HTTP Cookies

Les cookies HTTP sont de petits blocs de données conservés sur l'ordinateur de l'utilisateur par le navigateur Web. Elles sont disponibles aux développeurs pour stocker des informations qui vont être inclus avec chaque requête de l'utilisateur.

7.5 Mécanismes d'authentification

Nous utiliserons JWT comme mécanisme d'échange d'informations d'identification. Bien que nous disposions d'un mécanisme robuste pour vérifier l'intégrité des données, le stockage des jetons et leur échange en toute sécurité sur Internet n'est pas spécifié par la norme JWT, nous devons donc concevoir notre propre processus pour établir l'authentification et persister son état à travers les sessions.

7.5.1 Access token

Un jeton d'accès est un JWT qui contient les informations d'identification de l'utilisateur dans sa charge utile. Si ce jeton est attaché à l'en-tête d'une requête, l'accès est accordé et les requêtes sont transmises au middleware suivant. Un jeton d'accès a une période d'expiration relativement courte (dix minutes) après laquelle le jeton a expiré et n'est plus valide. Dans notre application client, ce jeton est enregistré dans une variable qui serait définie au moment de l'exécution pour éviter qu'il ne soit divulgué.

7.5.2 Refresh token

Un jeton d'actualisation est un autre JWT que nous utilisons pour l'authentification. La charge utile de ce jeton contient des informations sur l'appareil pour laquelle il est émis. Nous utilisons ce jeton pour actualiser les jetons d'accès des utilisateurs, conserver l'état d'authentification entre les sessions, gérer les appareils authentifiés et forcer la déconnexion des utilisateurs. Les jetons d'actualisation n'accordent l'accès à aucune ressource, ils peuvent donc être conservés dans le navigateur de l'utilisateur dans un cookie sécurisé.

7.5.3 Processus de connexion

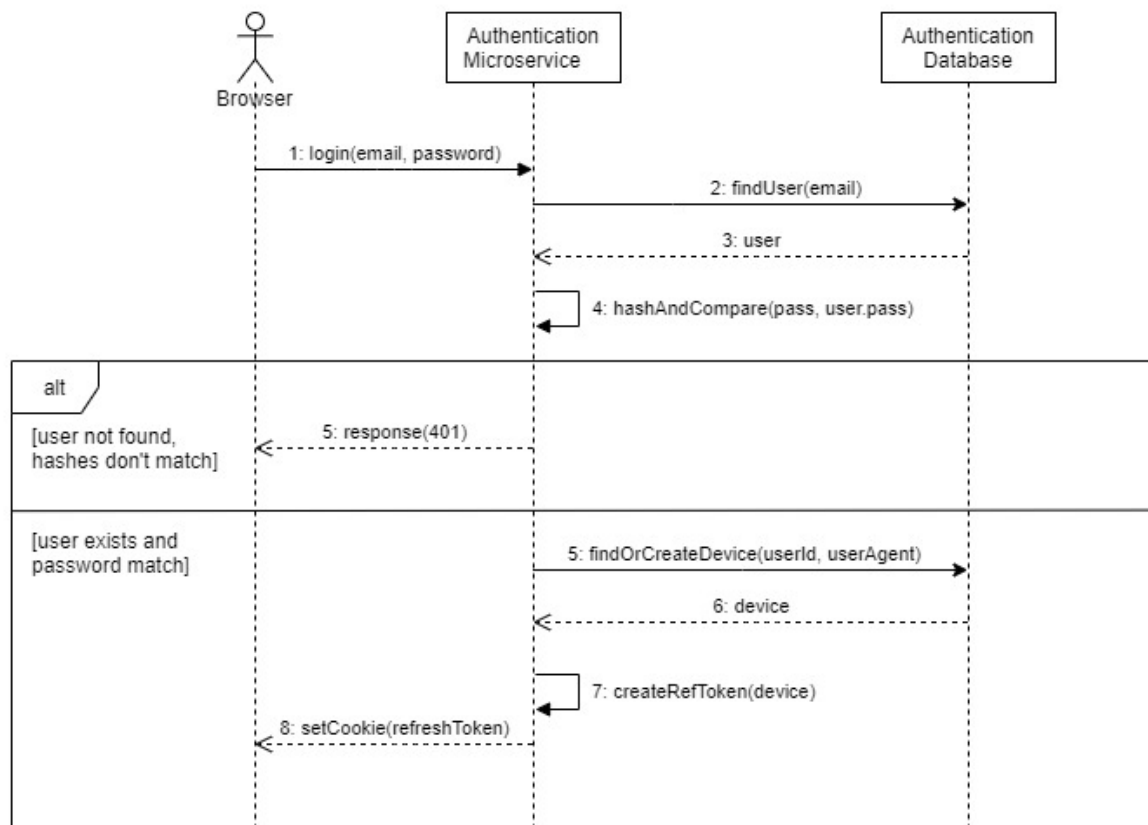


FIG. 7.3 : Diagramme de séquence de la connexion

La figure 7.3 explique le processus de connexion (login) : Lorsqu'un appel d'authentification avec un e-mail et un mot de passe est initié par le client, le service d'authentification commence d'abord par vérifier l'existence du compte. Si l'utilisateur n'est pas trouvé, une réponse HTTP avec le code d'état 403 pour indiquer une erreur dans les informations d'identification. Si l'utilisateur existe, nous comparons le hash enregistré du mot de passe contre le hash du mot de passe dans la requête, s'ils sont égaux, le processus se poursuit en récupérant ou en créant un appareil avec l'ID utilisateur et l'identificateur du navigateur. Cet appareil est utilisé pour créer le jeton d'actualisation qui sera ensuite utilisé pour actualiser le jeton d'accès.

7.5.4 Processus d'actualisation de jeton d'accès

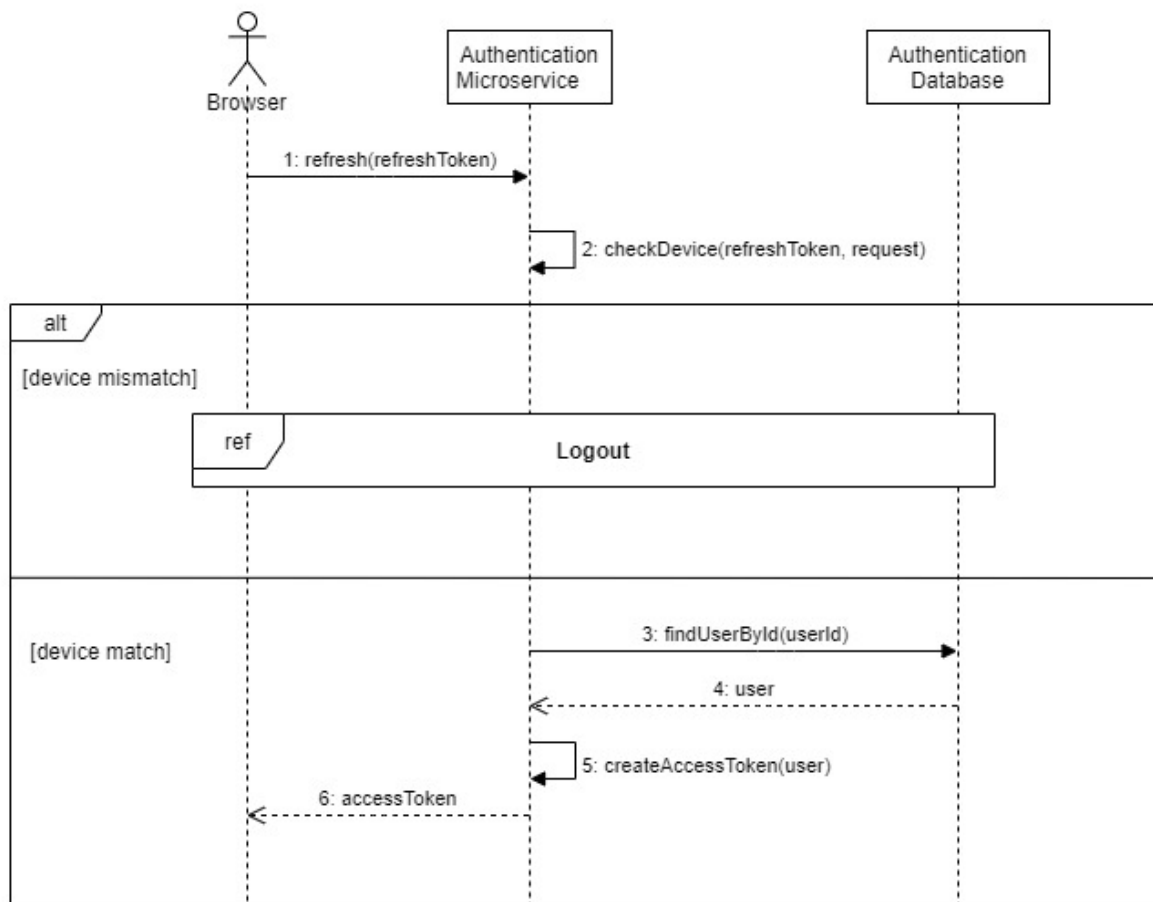


FIG. 7.4 : Diagramme de séquence d'actualisation de jeton d'accès

Comme l'illustre la figure 7.4, l'utilisateur commence par faire appel à un endpoint d'actualisation tout en incluant le cookie qui contient le jeton d'actualisation. Le service d'authentification commence par vérifier l'appareil d. Si les appareils ne correspondent pas, cela signifie que les cookies sont invalide et le flux de l'opération ne devrait pas se poursuivre. Si les appareils correspondent, le processus se poursuit en récupérant les informations de l'utilisateur, en créant un jeton d'accès et en le renvoyant.

7.5.5 Processus de validation de requêtes

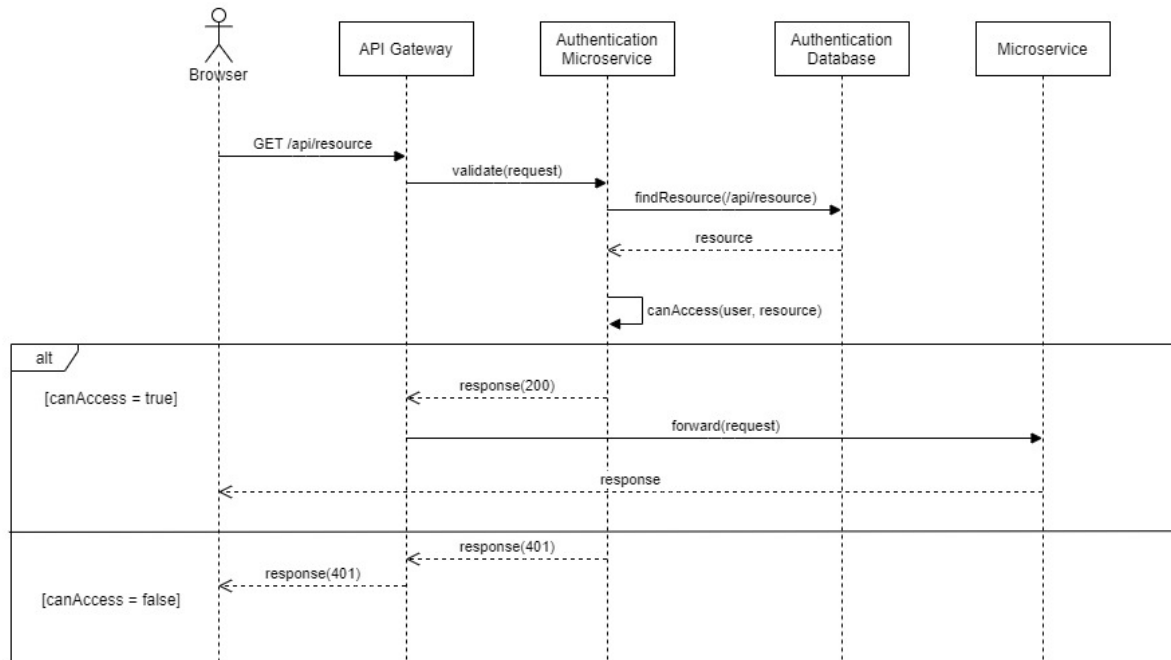


FIG. 7.5 : Diagramme de séquence de validation de requête

Lors de la réception d'une nouvelle requête, la passerelle API la redirige d'abord vers le micro-service d'authentification, plus précisément vers un endpoint qui autorise ou refuse l'accès en fonction du rôle de l'utilisateur et de la ressource souhaitée. Le service d'authentification tentera de récupérer une ressource par son chemin (path) unique, récupérera les rôles autorisés pour cette ressource et vérifiera enfin si le rôle de l'utilisateur est inclus dans la liste des rôles autorisés. Si l'utilisateur n'est pas autorisé, une réponse HTTP avec le code d'état 401 (Non autorisé) est renvoyée à la passerelle API, cela empêchera la requête de continuer. Cependant, si l'utilisateur est autorisé à accéder à cette ressource, l'authentification renvoie une réponse avec le code d'état 200 (OK) et la passerelle API transmet la demande au micro-service correspondant.

7.5.6 Processus de déconnexion

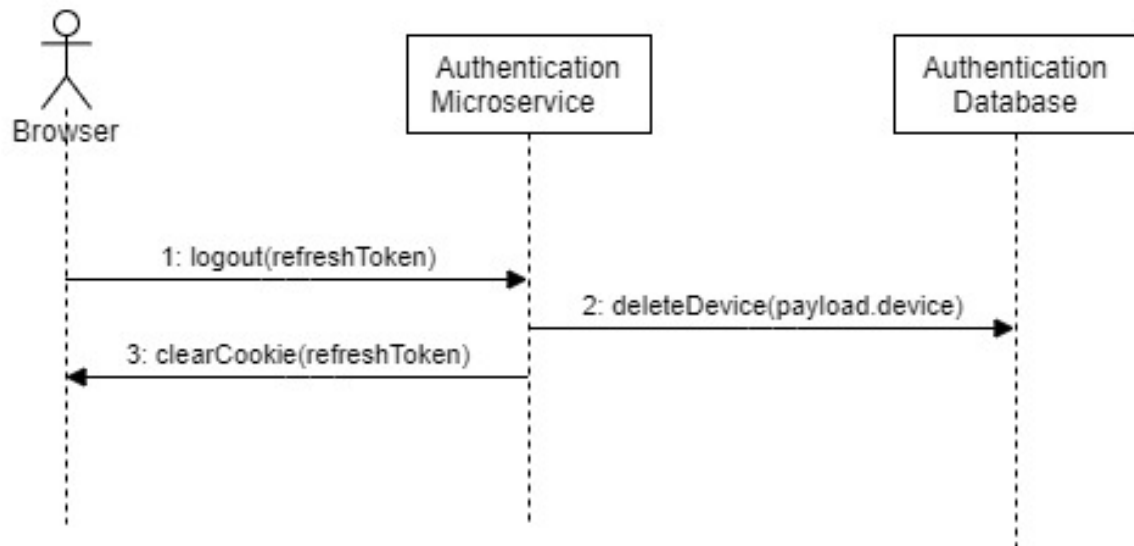


FIG. 7.6 : Diagramme de séquence de déconnexion

La déconnexion d'un utilisateur consiste à supprimer l'entrée de périphérique correspondante de la base de données et à effacer le cookie qui contient le jeton d'actualisation désormais invalide.

7.6 Atténuation des attaques

Nous allons discuter comment nous atténuons les deux attaques courantes sur Internet qui ciblent la façon dont les informations d'identification sont enregistrées.

7.6.1 Cross Site Scripting (XSS)

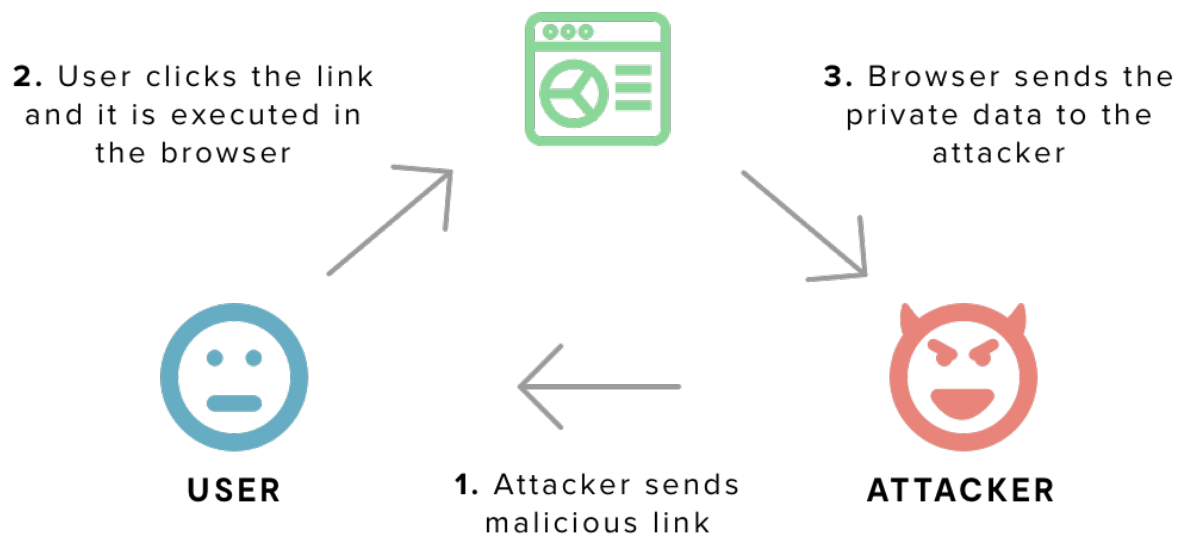


FIG. 7.7 : Les étapes d'une attaque XSS

Le Cross Site Scripting est l'une des attaques les plus courantes sur le Web. Elle commence lorsque l'utilisateur visite un site Web vulnérable ou malveillant, qui permet à un attaquant d'exécuter du code JavaScript malveillant sur le navigateur de l'utilisateur. En général, le code malveillant volerait les informations d'identification et les enverrait à l'attaquant. Dans notre service d'authentification, nous choisissons de ne pas enregistrer les jetons d'accès n'importe où et lorsque nous enregistrons des jetons d'actualisation dans les cookies, nous définissons des indicateurs spéciaux (`same-site=strict` et `httpOnly=true`) qui les rendent inaccessibles depuis JavaScript.

7.6.2 Cross-site Request Forgery (CSRF)

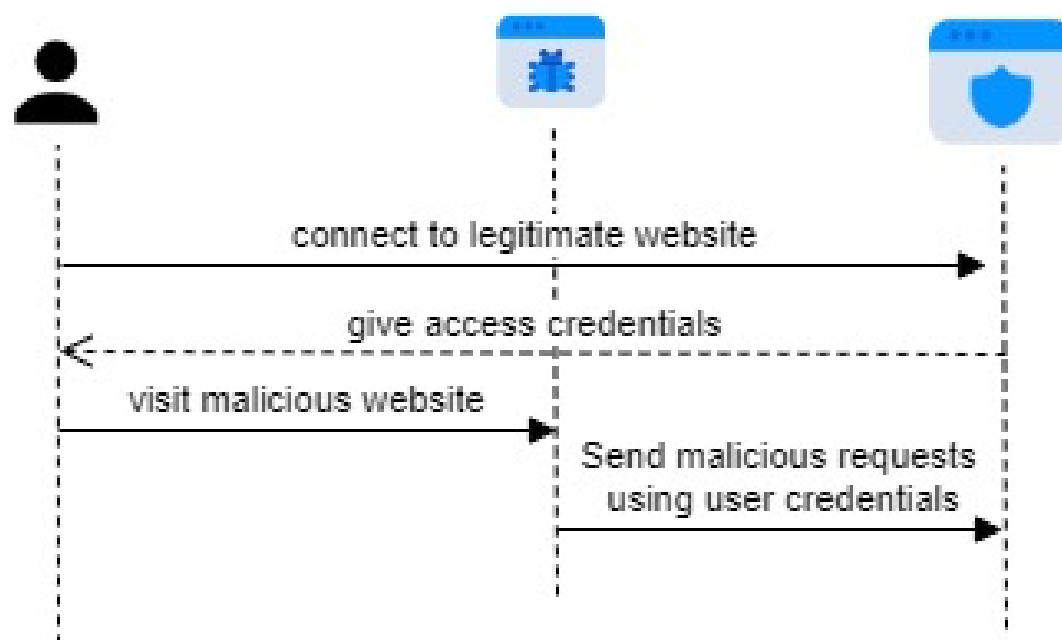


FIG. 7.8 : Les étapes d'une attaque CSRF

Cross-Site Request Forgery est une attaque qui exploite la façon dont les informations d'authentification sont enregistrées sur Internet. Cela commence également lorsque l'utilisateur visite un site Web malveillant après établir une connexion avec un site Web légitime (par exemple, un site Web bancaire). Le site Web malveillant essaye d'envoyer des requêtes au nom de l'utilisateur en utilisant ses informations d'identification, généralement enregistrés en cookies. Les demandes malveillantes peuvent être des appels pour modifier les informations d'identification, faire des transactions, etc. Par contre, le site attaquant n'aura pas accès aux réponses retournés puisque les requêtes ont été initiées par le navigateur de l'utilisateur. Dans notre cas, nous enregistrons uniquement le jeton d'actualisation dans les cookies, les jetons d'actualisation n'accordent l'accès à aucune ressource sauf l'actualisation du jeton d'accès.

7.7 Implémentation

7.7.1 Environnement de développement

Nous avons décidé de développer notre micro-service en NodeJS car la plupart des opérations qu'on fait sont lourdes en E/S (beaucoup de lecture de base de données) et demandent peu de puissance calcul. NodeJS est idéal pour ces types de workflows en raison de son architecture asynchrone basée sur les événements.

Nous profiterons également du riche écosystème de Node pour accélérer le développement

- **PassportJS** : Passport[20] est un middleware d'authentification pour NodeJS. Il supporte un ensemble complet de stratégies prenant en charge l'authentification à l'aide d'un nom d'utilisateur et d'un mot de passe, d'un serveur LDAP, de fournisseurs OAuth2, et beaucoup d'autres méthodes.
- **Bcrypt** : Bcrypt[21] est un algorithme de hachage de mot de passe. Il s'agit d'une fonction adaptative où le nombre d'itérations peut être augmenté par programmation pour le rendre plus lent, garantissant qu'il reste résistant aux attaques brute-force.
- **OpenAPI** : La spécification OpenAPI[22] définit une interface standard, indépendante du langage, pour les API REST qui nous permet de découvrir et de comprendre les capacités du service sans avoir besoin d'accéder au code source. Lorsqu'il est correctement défini, un consommateur peut comprendre et interagir avec le service distant avec une logique de mise en œuvre minimale.

7.7.2 Réalisation

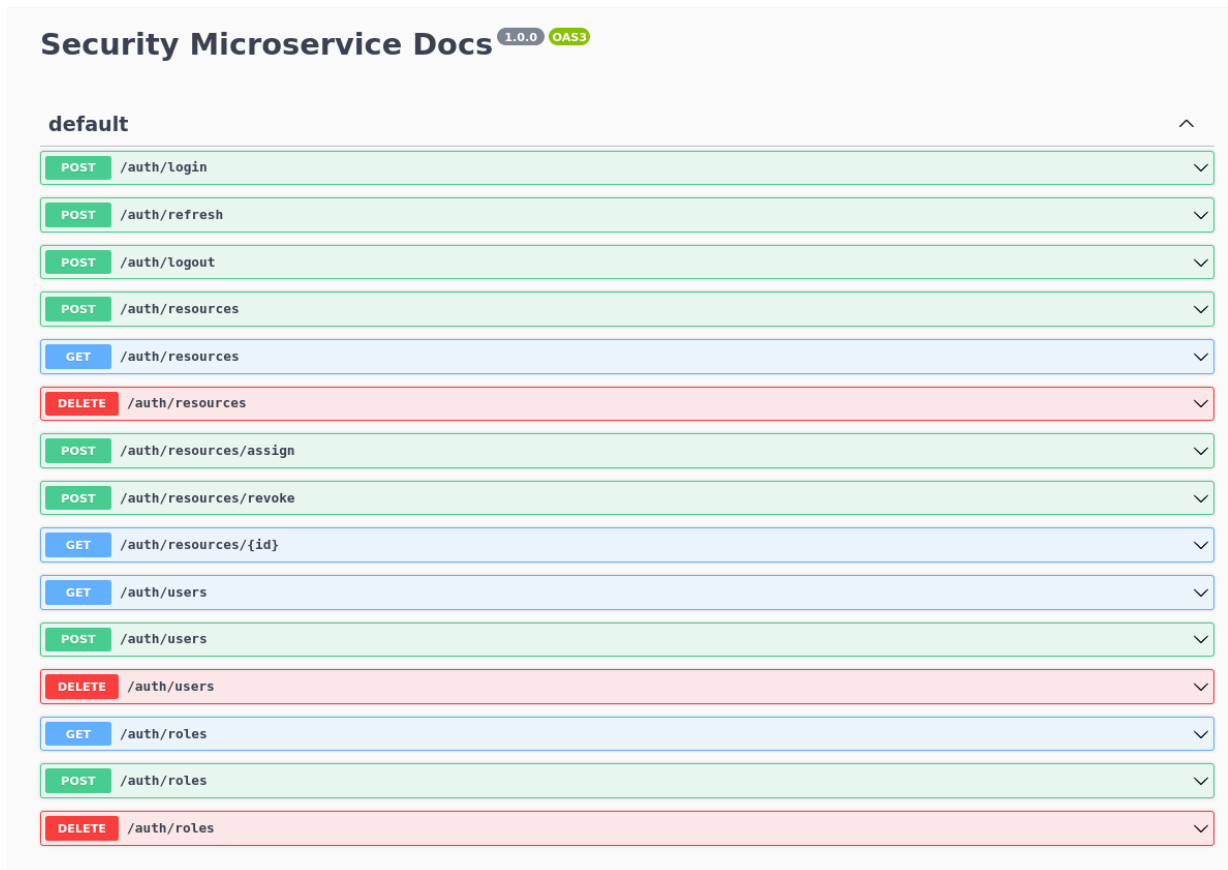


FIG. 7.9 : Fonctionnalités offertes par le service d'authentification

La figure 7.9 est une capture d'écran de l'interface de Swagger, l'outil qu'on utilise pour visualiser les données du protocole OpenAPI.

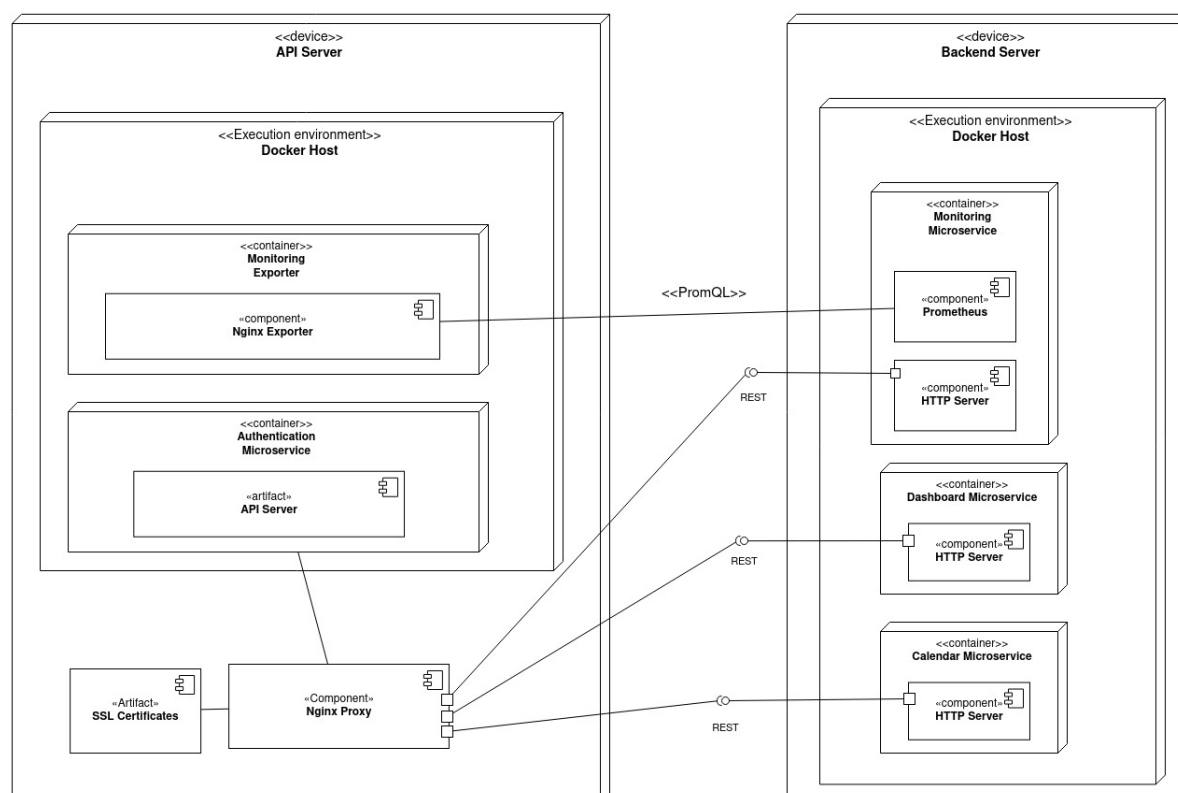


FIG. 7.10 : Diagramme de déploiement partiel de la plateforme

La figure 7.10 est le diagramme de déploiement d'une partie de la plateforme.

Notez bien que le micro-service de sécurité et Nginx sont déployées sur le même serveur. Étant donné que chaque demande à un service de backend privé passe d'abord par le micro-service d'authentification, son déploiement sur un autre serveur augmenterait la latence. Nous déployons également un exportateur Nginx pour surveiller la passerelle API et obtenir des informations sur ses performances.

Conclusion

Nous avons abordé les problèmes de sécurisation d'une architecture de micro-services dans ce chapitre. Nous avons démontré comment les outils et les approches que nous avons utilisées pour atteindre notre objectif peuvent résister aux attaques les plus courantes contre le stockage des informations d'identification en ligne. Enfin, nous avons présenté les détails de mise en œuvre de notre service.

Conclusion Générale

Ce rapport détaille notre travail de quatre mois chez Business & Ai lors de notre stage de fin d'études, avec l'objectif d'obtenir du diplôme national d'ingénieur en informatique. Pendant ce temps, nous avons pu mettre à profit nos connaissances théoriques acquis à l'ISSAT Sousse en participant à la conception et à la mise en œuvre d'une plateforme d'analyse de trésorerie distribuée construite sur l'architecture micros-ervice et pilotée par des agents d'intelligence artificielle.

Nous avons commencé notre projet en envisageant la solution optimale que nous voulons construire. Au cours de cette phase, nous avons expérimenté avec plusieurs technologies pour concevoir l'architecture globale que nous développerions pour le reste de notre stage.

Comme le traitement des données est la pièce centrale dans le système, nous avons naturellement commencé par la développer. À l'aide d'Apache Airflow, nous avons construit un pipeline de traitement de données adaptable et évolutif. Nous avons profité des capacités intégrées du framework pour accélérer le développement et garantir que notre solution répond aux exigences de l'industrie. Ensuite nous avons

Par la suite, nous avons concentré nos efforts sur la création d'un tableau de bord de visualisation des données. À l'aide d'un système de cache implémentée dans Redis, une base de données en mémoire, nous avons conçu une technique pour réduire la charge sur la base de données centrale.

Nous avons inclus les pratiques DevOps dans notre flux de travail dès le début du développement de tous les services. Nous avons pu construire des pipelines CI/CD pour chaque sous-projet à l'aide de Git, Docker, GitHub et DockerHub. Cela nous a permis de déployer le logiciel rapidement tout en garantissant la qualité du logiciel par le biais de tests unitaires, test d'intégration et end-to-end tests.

Nous voulions un mécanisme pour surveiller le comportement de nos applications dans un environnement de production une fois que nous les avons déployées. L'étape suivante consistait donc à créer le service de surveillance de l'infrastructure. Nous avons commencé par rechercher des solutions actuelles et sélectionner celle qui nous convenait le mieux. Nous avons fini par combiner Prometheus et Grafana pour créer un système de surveillance en temps réel qui pourrait être facilement étendu et modifié.

Nous sommes tous conscients de la nécessité de protéger nos applications contre les attaques aléatoires sur Internet. Notre prochaine étape consistait à ajouter une couche d'authentification et d'autorisation à l'ensemble de la plate-forme.

Enfin, notre projet est loin d'être terminé. Il y a encore des aspects que nous pourrions

améliorer et développer davantage, ainsi que d'autres que nous n'avons pas eu assez de temps pour aborder, comme la construction de l'ensemble de la plate-forme dans un cluster Kubernetes, ce qui nous donnerait plus de flexibilité dans la façon dont nous déployons nos applications et profitez des services de mise à l'échelle automatique des fournisseurs de cloud.

References

- [1] *Monolithic architecture vs microservices*. Visitée le 05/07/2021. URL : <https://www.divante.com/blog/monolithic-architecture-vs-microservices>.
- [2] *Agile Manifesto*. Visitée le 05/07/2021. URL : <https://agilemanifesto.org/>.
- [3] *What is SCRUM*. Visitée le 05/07/2021. URL : <https://www.scrum.org/resources/what-is-scrum>.
- [4] *Extreme Programming (XP)*. Visitée le 05/07/2021. URL : <https://www.agilealliance.org/glossary/xp>.
- [5] *What Is Rapid Application Development (RAD)*. Visitée le 05/07/2021. URL : <https://blog.capterra.com/what-is-rapid-application-development/>.
- [6] *Spotify Luigi GitHub repository*. Visitée le 05/07/2021. URL : <https://github.com/spotify/luigi>.
- [7] *Airflow*. Visitée le 05/07/2021. URL : <https://airflow.apache.org/>.
- [8] *Apache Oozie Workflow Scheduler for Hadoop*. Visitée le 05/07/2021. URL : <https://oozie.apache.org/>.
- [9] *Solid OOP Design Principles*. Visitée le 05/07/2021. URL : https://www.digitalocean.com/community/conceptual_articles/s-o-l-i-d-the-first-five-principles-of-object-oriented-design.
- [10] *Git Branches*. Visitée le 05/07/2021. URL : <https://www.nobledesktop.com/learn/git/git-branches>.
- [11] *DOCKERHUB*. Visitée le 05/07/2021. URL : <https://hub.docker.com/>.
- [12] *FLYWAYDB*. Visitée le 05/07/2021. URL : <https://flywaydb.org/>.
- [13] *NAGIOS. Nagios*. Visitée le 05/07/2021. URL : <https://www.nagios.org/>.
- [14] *DataDog*. Visitée le 05/07/2021. URL : <https://www.datadoghq.com/>.
- [15] *Sentry*. URL : <https://sentry.io/welcome/>.
- [16] *Prometheus*. Visitée le 05/07/2021. URL : <https://prometheus.io/>.
- [17] *Grafana*. Visitée le 05/07/2021. URL : <https://grafana.com/>.
- [18] *JSON Web Tokens*. Visitée le 05/07/2021. URL : <https://jwt.io/>.

- [19] *RSA Algorithm in Cryptography*. Visitée le 05/07/2021. URL : <https://www.geeksforgeeks.org/rsa-algorithm-cryptography/>.
- [20] *Simple, unobtrusive authentication for Node.js*. Visitée le 05/07/2021. URL : <http://www.passportjs.org/>.
- [21] *Hashing in Action : Understanding bcrypt*. Visitée le 05/07/2021. URL : <https://auth0.com/blog/hashing-in-action-understanding-bcrypt/>.
- [22] *OpenAPI Specification*. Visitée le 05/07/2021. URL : <https://swagger.io/specification/>.

Resumé

Ce projet établit une architecture micro-services pour aider à la création d'une plateforme d'analyse de trésorerie. Tout au long de notre stage, nous avons développé les services de base de la plateforme. Puisque notre plate-forme est dirigée par les données, nous avons commencé par créer nos pipelines de traitement des données dans Airflow. De plus, nous avons utilisé Git, GitHub, DockerHub et SSH pour créer des pipelines CI/CD qui automatiseraient les tests et le déploiement de nos applications. Nous avons créé un tableau de bord analytique qu'on utilisera pour explorer les données d'une manière significative. De plus, nous avons mis en place un service de surveillance du matériel pour surveiller le comportement de notre plate-forme dans un environnement de production et repérer les problèmes possibles. Enfin, nous avons terminé le stage en créant un service d'authentification et d'autorisation qui sécuriseraient notre plateforme contre les accès non identifiés

Mots clés : Micro-services, Docker, Architecture orientée systèmes, DevOps, CI/CD, NodeJS, Airflow, Angular

